INTEGRATIVE PROGRAMMING

ACTIVITY COMPILATION

Activities 1 to 13 - Visual Basic .NET Windows Forms and Console Programs

Submitted by : Mangalili, Fhetz

Course, Year and Section : BSIT 3B

Subject : Integrative Programming

Submitted to : _____

School Year : School Year 2026 - 2027

Date Submitted : October 26, 2026

This compilation contains the thirteen activities of the semester. The form designs of Activities 4, 5, 6, 8, 10 and 12 were newly drawn and their program code was rewritten, while the names of the author, the course and the program titles were updated in every project.

Contents

Activity 1 - Console Basics and Data Types 3

Activity 2 - Name Input and Celsius - Fahrenheit Converter 4

Activity 3 - Sum, Difference, Product, Quotient and Average (Console with Retry) 5

Activity 4 - Arithmetic Machine (Windows Form) 7

Activity 5 - Ternary Remarks (If Operator) 10

Activity 6 - Select Case Grade Evaluation 13

Activity 7 - Integrative Program 1 - Interest, Square and Class Average 16

Activity 8 - The Message (MessageBox Options) 19

Activity 9 - ListView File Explorer 22

Activity 10 - Panel and SplitContainer Laboratory 25

Activity 11 - Windows Forms Controls Collection (TabControl, TreeView, ListView, Dialogs) 32

Activity 12 - File Explorer Clone (PanelSplitDemo) 38

Activity 13 - Integrative Project - School Records with MDI Forms 47

Appendix A - Project file list

Appendix B - What changed in this compilation

Activity 1

Activity 1 - Console Basics and Data Types

Project : Act1_Activity202606241600 | Author : Mangalili, Fhetz | BSIT 3B

Description

The first activity is a console application that prints a short greeting and declares one variable of every basic data type (String, Integer, Double, Boolean and Char). The program ends with Console.ReadKey() so the console window stays open until a key is pressed.

Objectives

- Create a Visual Basic .NET console application and identify the Module and Sub Main entry point.
- Declare variables with the correct data type and assign initial values.
- Print output with Console.WriteLine and pause the console with Console.ReadKey.

Source Code

Module1.vb

```
Module Module1

    Sub Main()
        Console.WriteLine("Activity202606241600")
        Console.WriteLine("Author: Mangalili, Fhetz")

        Dim theName = "Mangalili, Fhetz"
        Console.WriteLine("Hello " + theName)

        Dim n1 As Integer = 0
        Dim n2 As Double = 0.0
        Dim isValid As Boolean = True
        Dim character As Char = "A"
        Dim number1 As Integer = 12
        Dim number2 As Double = 12.12

        Console.ReadKey()
    End Sub

End Module
```

Sample Output

```
Activity202606241600
Author: Mangalili, Fhetz
Hello Mangalili, Fhetz

Press any key to continue . . .
```

Activity 2

Activity 2 - Name Input and Celsius - Fahrenheit Converter

Project : Act2 Celcius-Fahrenheit Converter | Author : Mangalili, Fhetz | BSIT 3B

Description

This console program asks for the last name and the first name of the user, prints them back, and then performs two temperature conversions. The first conversion changes Celsius into Fahrenheit using the formula (C x 1.8) + 32, and the second conversion changes Fahrenheit into Celsius using (F - 32) / 1.8.

Objectives

- Read input from the user with Console.ReadLine.
- Convert text input into numbers with Convert.ToDecimal.
- Apply arithmetic formulas and display the computed results.

Source Code

Module1.vb

```
Module Module1

    Sub Main()
        Console.Write("Hello world")
        Console.ReadLine()

        Dim lastName As String = ""
        Dim firstName As String = ""

        Console.Write("Enter Lastname: ")
        lastName = Console.ReadLine()
        Console.Write("Enter Firstname: ")
        firstName = Console.ReadLine()

        Console.WriteLine("Your Lastname is " + lastName)
        Console.WriteLine("Your Firstname is " + firstName)
        Console.ReadLine()

        ' Converter 1: Celsius to Fahrenheit
        Console.WriteLine("--- Celsius to Fahrenheit Converter ---" & vbNewLine)
        Console.Write("Enter Celsius: ")
        Dim celsiusInput As Decimal = Convert.ToDecimal(Console.ReadLine())
        Dim fahrenheitResult As Decimal = (celsiusInput * 1.8) + 32
        Console.WriteLine("Result in Fahrenheit: " & fahrenheitResult)
        Console.ReadLine()

        ' Converter 2: Fahrenheit to Celsius
        Console.WriteLine("--- Fahrenheit to Celsius Converter ---" & vbNewLine)
        Console.Write("Enter Fahrenheit: ")
        Dim fahrenheitInput As Decimal = Convert.ToDecimal(Console.ReadLine())
        Dim celsiusResult As Decimal = (fahrenheitInput - 32) / 1.8
        Console.WriteLine("Result in Celsius: " & celsiusResult)
        Console.ReadLine()

    End Sub

End Module
```

Sample Output

```
Hello world
Enter Lastname: Mangalili
Enter Firstname: Fhetz
Your Lastname is Mangalili
Your Firstname is Fhetz
--- Celsius to Fahrenheit Converter ---
Enter Celsius: 37
Result in Fahrenheit: 98.6
--- Fahrenheit to Celsius Converter ---
Enter Fahrenheit: 98.6
Result in Celsius: 37.0
```

Activity 3

Activity 3 - Sum, Difference, Product, Quotient and Average (Console with Retry)

Project : Activity3_20260629 | Author : Mangalili, Fhetz | BSIT 3B

Description

The program accepts three whole numbers and displays their sum, difference, product, quotient and average. A Try - Catch block with the label ulit traps a FormatException, so when the user types a letter instead of a number the program prints the error message and asks for the three numbers again instead of closing.

Objectives

- Use Convert.ToInt16 to read three numbers.
- Compute the five required results and display them with labels.
- Handle invalid input with Try - Catch and a GoTo label so the program repeats.

Source Code

Module1.vb

```
Module Module1
    Sub Main()

        'Activity3_20260629'
        'Create a vb.net console app that will accept 3 numbers, and display the sum, difference, product,
    quotient, and ave of the 3 numbers'

        Try
ulit:
            Console.WriteLine("Activity3_20260629")
            Console.WriteLine("Author: Mangalili, Fhetz")
            Console.WriteLine("Course Year Section: BSIT 3B")

            Console.Write("Enter number 1: ")
            Dim n1 As Integer = Convert.ToInt16(Console.ReadLine())
            Console.Write("Enter number 2: ")
            Dim n2 As Integer = Convert.ToInt16(Console.ReadLine())
            Console.Write("Enter number 3: ")
            Dim n3 As Integer = Convert.ToInt16(Console.ReadLine())

            Console.Write("The sum is: ")
            Dim sum As Double = n1 + n2 + n3
            Console.WriteLine(sum)

            Console.Write("The difference is: ")
            Dim difference As Double = n1 - n2 - n3
            Console.WriteLine(difference)

            Console.Write("The product is: ")
            Dim product As Double = n1 * n2 * n3
            Console.WriteLine(product)

            Console.Write("The quotient is: ")
            Dim quotient As Double = n1 / n2 / n3
            Console.WriteLine(quotient)

            Console.Write("The average is: ")
            Dim ave As Double = sum / 3
            Console.WriteLine(ave)

        Catch ex As FormatException
            Console.WriteLine("Error: " & ex.Message)
            GoTo ulit
        End Try

        Console.ReadKey()

        'Iti nasursurwak ditoy nga taraon ket iti makuna nga "convertion method" iti agduduma nga
    datatypes.'
        'ken syak ket na gwapo'
    End Sub
End Module
```

Sample Output

```
Activity3_20260629
Author: Mangalili, Fhetz
Course Year Section: BSIT 3B
Enter number 1: 10
Enter number 2: 5
Enter number 3: 2
The sum is: 17
The difference is: 3
The product is: 100
The quotient is: 1
The average is: 5.666666666666667
```

Activity 4

Activity 4 - Arithmetic Machine (Windows Form)

Project : Activity4_20260703 | Author : Mangalili, Fhetz | BSIT 3B

Description

Activity 4 converts the console computation of Activity 3 into a Windows Form application. The form is divided into an input card, a result card and a row of action buttons. The three numbers are read once by a shared function, and every operation button writes its answer in its own result box.

The interface of this activity was redesigned for this compilation: the old free-floating labels and text boxes were replaced by two bordered panels, a dark banner that carries the activity number and the author, and colour coded buttons. The program logic was rewritten as well: a single ReadInputs() function validates the entries, a ShowResult() routine formats every answer to four decimal places, and the quotient button checks for division by zero before it computes.

Objectives

- Design a Windows Form with Panel containers, Labels, TextBoxes and Buttons.
- Declare module level variables and use a Function that returns Boolean for validation.
- Trap conversion errors with Try - Catch and guard the quotient against division by zero.

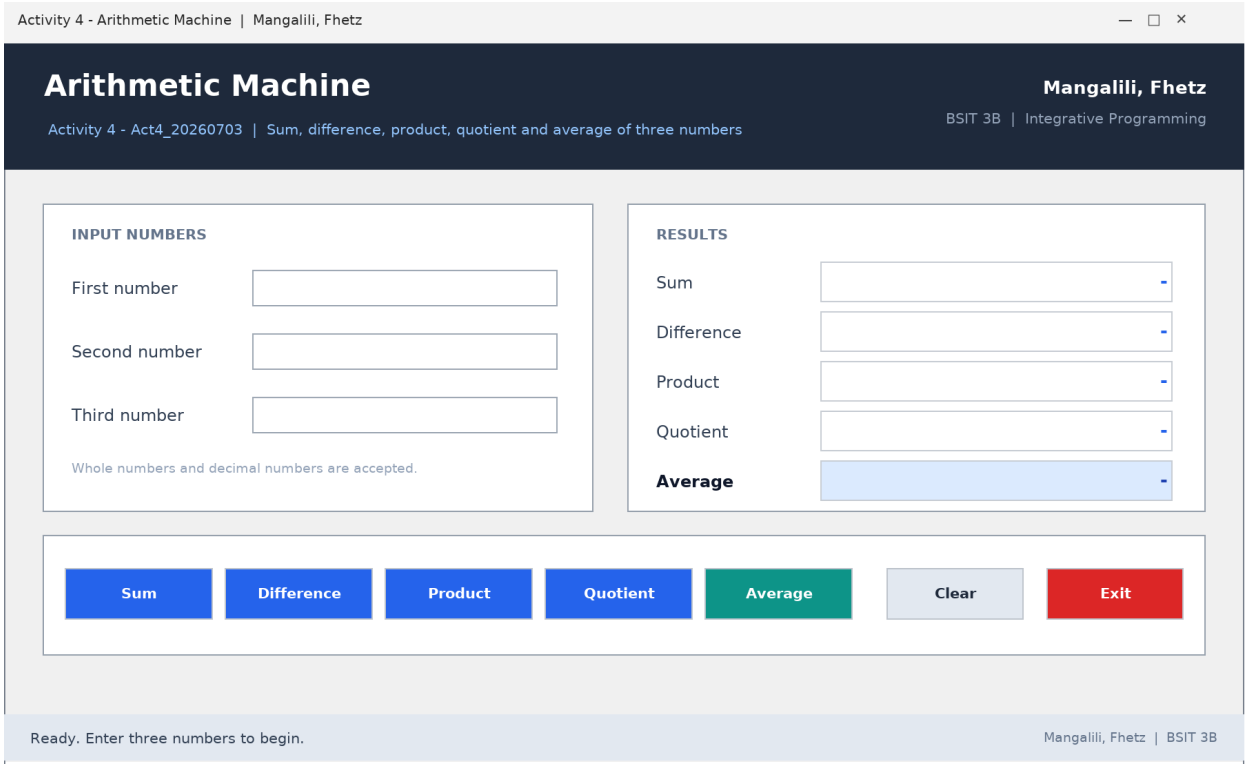


Figure 4.1 - Form design of Activity 4 - Arithmetic Machine

Source Code

Form1.vb

```
' ============================================================
' Activity 4 - Act4_20260703
' Arithmetic Machine : sum, difference, product, quotient and
' average of three numbers entered by the user.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
Public Class Form1

    Private number1 As Double
```

```
Private number2 As Double
Private number3 As Double
Private inputsAreValid As Boolean

Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
    ResetForm()
    txtN1.Focus()
End Sub

' Reads the three text boxes and stores the values in the
' module level variables. Returns False when an entry is not a number.
Private Function ReadInputs() As Boolean
    Try
        number1 = Convert.ToDouble(txtN1.Text.Trim())
        number2 = Convert.ToDouble(txtN2.Text.Trim())
        number3 = Convert.ToDouble(txtN3.Text.Trim())
        inputsAreValid = True
    Catch ex As Exception
        inputsAreValid = False
        ClearResults()
        lblStatus.Text = "Invalid entry. Please type numbers only (for example 12 or 12.5)."
    End Try
    Return inputsAreValid
End Function

Private Sub ShowResult(ByVal target As Label, ByVal value As Double, ByVal operation As String)
    target.Text = value.ToString("N4")
    lblStatus.Text = operation & " = " & value.ToString("N4")
End Sub

Private Sub ClearResults()
    lblSumValue.Text = "-"
    lblDiffValue.Text = "-"
    lblProdValue.Text = "-"
    lblQuotValue.Text = "-"
    lblAveValue.Text = "-"
End Sub

Private Sub ResetForm()
    txtN1.Clear()
    txtN2.Clear()
    txtN3.Clear()
    ClearResults()
    lblStatus.Text = "Ready. Enter three numbers to begin."
    txtN1.Focus()
End Sub

Private Sub btnSum_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnSum.Click
    If Not ReadInputs() Then Return
    ShowResult(lblSumValue, number1 + number2 + number3, "Sum")
End Sub

Private Sub btnDifference_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnDifference.Click
    If Not ReadInputs() Then Return
    ShowResult(lblDiffValue, number1 - number2 - number3, "Difference")
End Sub

Private Sub btnProduct_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnProduct.Click
    If Not ReadInputs() Then Return
    ShowResult(lblProdValue, number1 * number2 * number3, "Product")
End Sub

Private Sub btnQuotient_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnQuotient.Click
    If Not ReadInputs() Then Return
    If number2 = 0 OrElse number3 = 0 Then
        lblQuotValue.Text = "undefined"
        lblStatus.Text = "Quotient is undefined because the second or third number is zero."
        Return
    End If
    ShowResult(lblQuotValue, number1 / number2 / number3, "Quotient")
End Sub

Private Sub btnAverage_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnAverage.Click
    If Not ReadInputs() Then Return
    ShowResult(lblAveValue, (number1 + number2 + number3) / 3, "Average")
End Sub

Private Sub btnClear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnClear.Click
    ResetForm()
End Sub
```

```
    Private Sub btnExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnExit.Click
        If MessageBox.Show("Close the Arithmetic Machine?", "Exit", _
                           MessageBoxButtons.YesNo, MessageBoxIcon.Question) = DialogResult.Yes Then
            Application.Exit()
        End If
    End Sub
End Class
```

Notes

Average is highlighted in blue because it is the summary value of the three operations above it. Clear returns the form to its start-up state, and Exit asks for confirmation before the application closes.

Activity 5

Activity 5 - Ternary Remarks (If Operator)

Project : Act5 - Ternary2 | Author : Mangalili, Fhetz | BSIT 3B

Description

The user types one average grade and the program displays three different remarks: the pass or fail remark, the letter grade and the numeric rating. Every result is produced by one nested ternary If expression instead of a long If - ElseIf block.

For this compilation the form was rebuilt as a two-column layout. The dark side rail holds the title and the name of the author, while the right side contains the entry box and three result cards. The code was reorganised into small procedures (ClearOutputs, ApplyCardColor), the Enter key inside the text box runs the evaluation, and the first card changes colour depending on the result.

Objectives

- Use the ternary If(condition, truePart, falsePart) operator.
- Nest several If operators to build a grading scale.
- Validate numeric input with Try - Catch and guide the user with status messages.

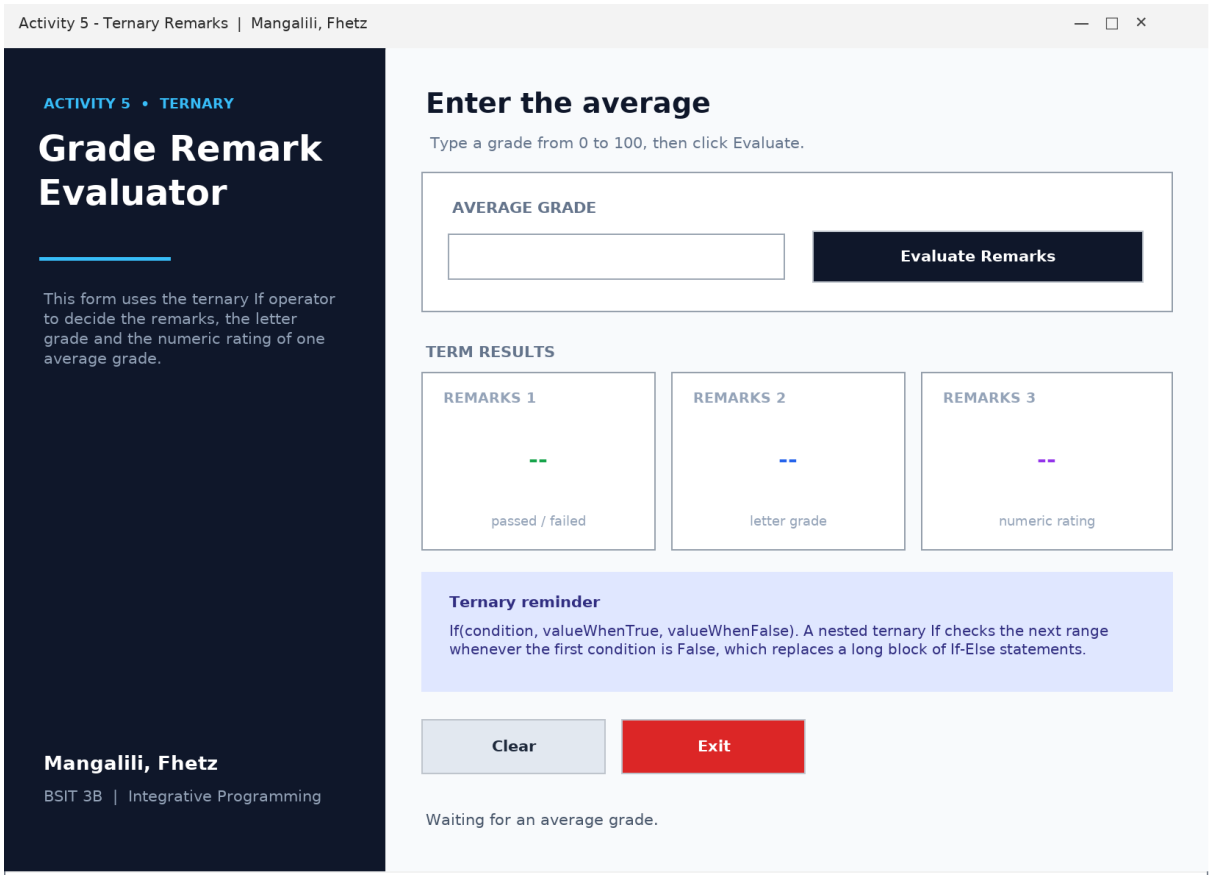


Figure 5.1 - Form design of Activity 5 - Ternary Remarks

Source Code

Form1.vb

```
' ============================================================
' Activity 5 - Ternary2
' Displays the remarks, the letter grade and the numeric rating
' of an average grade using nested ternary (If) operators.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
```

```
Public Class Form1

    Private averageGrade As Double

    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
        ClearOutputs()
        lblStatus.Text = "Waiting for an average grade."
        txtAverage.Focus()
    End Sub

    Private Sub ClearOutputs()
        lblCard1Value.Text = "--"
        lblCard2Value.Text = "--"
        lblCard3Value.Text = "--"
    End Sub

    Private Sub txtAverage_KeyPress(ByVal sender As Object, ByVal e As
    System.Windows.Forms.KeyPressEventArgs) Handles txtAverage.KeyPress
        ' Enter key inside the textbox works like clicking the Evaluate button
        If e.KeyChar = Microsoft.VisualBasic.ChrW(13) Then
            btnEvaluate.PerformClick()
            e.Handled = True
        End If
    End Sub

    Private Sub btnEvaluate_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnEvaluate.Click
        Try
            averageGrade = Convert.ToDouble(txtAverage.Text.Trim())

            ' Remarks 1 : passed / conditional / failed
            lblCard1Value.Text = If(averageGrade >= 75, "Passed",
                                 If(averageGrade >= 70, "Conditional", "Failed"))

            ' Remarks 2 : letter grade
            lblCard2Value.Text = If(averageGrade >= 94, "A",
                                 If(averageGrade >= 89, "B",
                                 If(averageGrade >= 84, "C",
                                 If(averageGrade >= 80, "D",
                                 If(averageGrade >= 75, "E", "F")))))

            ' Remarks 3 : numeric rating
            lblCard3Value.Text = If(averageGrade >= 98, "1.00",
                                 If(averageGrade >= 95, "1.25",
                                 If(averageGrade >= 92, "1.50",
                                 If(averageGrade >= 89, "1.75",
                                 If(averageGrade >= 86, "2.00",
                                 If(averageGrade >= 83, "2.25",
                                 If(averageGrade >= 80, "2.50",
                                 If(averageGrade >= 77, "2.75",
                                 If(averageGrade >= 75, "3.00",
                                 If(averageGrade >= 70, "5.00", "Invalid Grade...")))))))))

            If averageGrade > 100 OrElse averageGrade < 0 Then
                lblStatus.Text = "The grade " & averageGrade.ToString("0.##") & " is outside the 0 to 100
    range."
            Else
                lblStatus.Text = "Average " & averageGrade.ToString("0.##") & " evaluated using nested
    ternary If operators."
            End If

            ApplyCardColor(averageGrade)
        Catch ex As Exception
            ClearOutputs()
            lblCard1Value.Text = "Invalid"
            lblCard2Value.Text = "Invalid"
            lblCard3Value.Text = "Invalid"
            lblStatus.Text = "Please type a valid number, for example 88.5"
            txtAverage.SelectAll()
            txtAverage.Focus()
        End Try
    End Sub

    ' Small extra touch : the first card changes color with the result
    Private Sub ApplyCardColor(ByVal grade As Double)
        If grade >= 75 Then
            lblCard1Value.ForeColor = System.Drawing.Color.FromArgb(22, 163, 74)
        ElseIf grade >= 70 Then
            lblCard1Value.ForeColor = System.Drawing.Color.FromArgb(217, 119, 6)
        Else
            lblCard1Value.ForeColor = System.Drawing.Color.FromArgb(220, 38, 38)
        End If
    End Sub

    Private Sub btnClear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnClear.Click
        txtAverage.Clear()
```

```
        ClearOutputs()
        lblStatus.Text = "Cleared. Type another average grade."
        txtAverage.Focus()
    End Sub

    Private Sub btnExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnExit.Click
        Application.Exit()
    End Sub
End Class
```

Notes

Test values that show the boundaries: 75 (Passed / E / 3.00), 74 (Conditional / F / 5.00) and 98 (Passed / A / 1.00).

Activity 6

Activity 6 - Select Case Grade Evaluation

Project : Act6_IntegProg1_20260715 | Author : Mangalili, Fhetz | BSIT 3B

Description

Three grades are entered and the program computes their average. Three separate Select Case blocks then convert the average into the pass or fail remark, the letter grade and the numeric rating.

The form was redesigned for this compilation. The three grade boxes are grouped in one card on the left, the average is shown in a large dark badge on the right with three remark rows below it, and the badge changes colour (green, amber or red) depending on the result. The code was cleaned up so that the old duplicated button handlers are gone, validation is done by one ReadGrades() function, and grades outside the 0 to 100 range are rejected.

Objectives

- Compute the average of three grades.
- Use Select Case with relational cases (Case Is >=) and ranges (Case 70 To 74).
- Present the result with colour coded feedback.

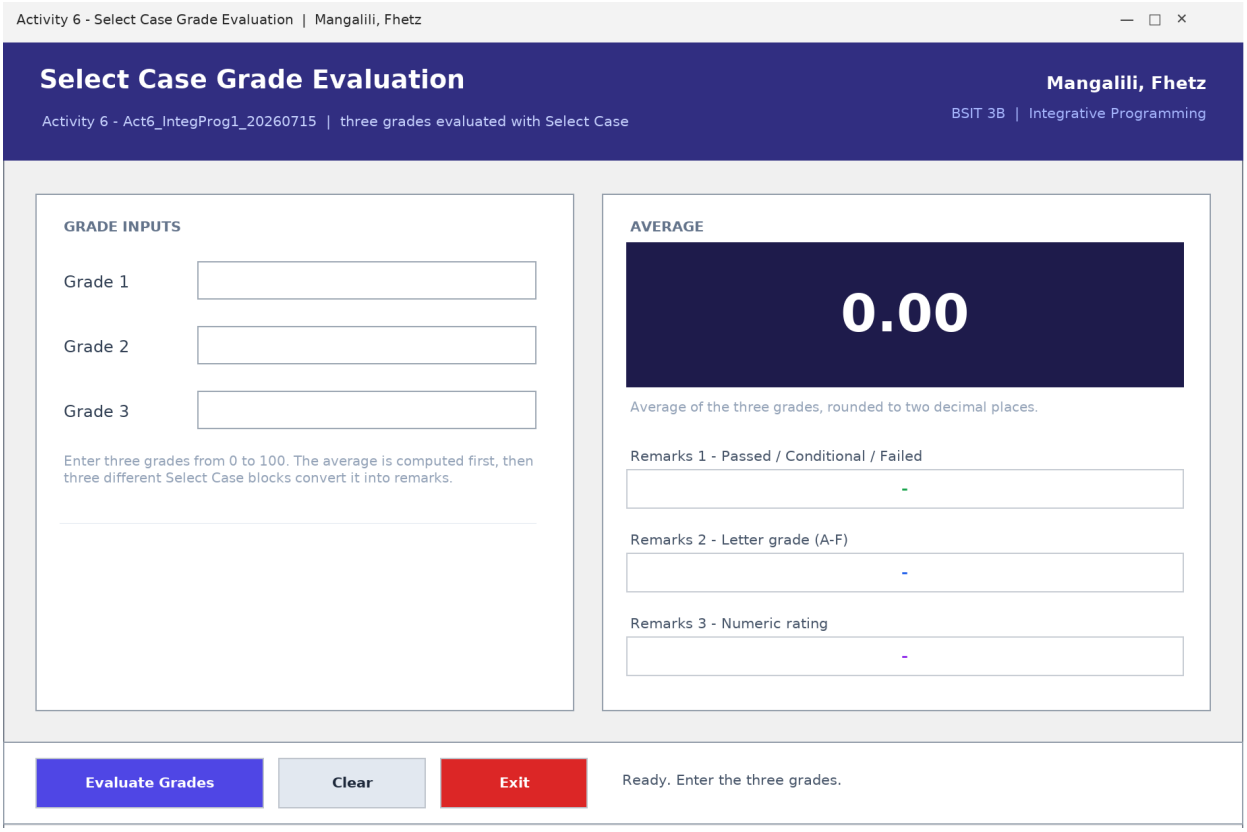


Figure 6.1 - Form design of Activity 6 - Select Case Grade Evaluation

Source Code

Form1.vb

```
' ============================================================
' Activity 6 - Act6_IntegProg1_20260715
' Reads three grades, computes the average and displays three
' different remarks using Select Case.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
Public Class Form1
```

```
Private grade1 As Double
Private grade2 As Double
Private grade3 As Double
Private averageGrade As Double

Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
    ResetScreen()
End Sub

Private Sub ResetScreen()
    txtG1.Clear()
    txtG2.Clear()
    txtG3.Clear()
    lblAverage.Text = "0.00"
    lblRemarks1.Text = "-"
    lblRemarks2.Text = "-"
    lblRemarks3.Text = "-"
    lblRemarks1.ForeColor = System.Drawing.Color.FromArgb(22, 163, 74)
    pnlAveBadge.BackColor = System.Drawing.Color.FromArgb(30, 27, 75)
    lblStatus.Text = "Ready. Enter the three grades."
    txtG1.Focus()
End Sub

Private Function ReadGrades() As Boolean
    Try
        grade1 = Convert.ToDouble(txtG1.Text.Trim())
        grade2 = Convert.ToDouble(txtG2.Text.Trim())
        grade3 = Convert.ToDouble(txtG3.Text.Trim())
    Catch ex As Exception
        lblAverage.Text = "0.00"
        lblRemarks1.Text = "Invalid entry"
        lblRemarks2.Text = "Numbers only"
        lblRemarks3.Text = "Numbers only"
        lblStatus.Text = "Only numeric grades are accepted. Example : 88 or 91.5"
        Return False
    End Try

    If grade1 < 0 OrElse grade2 < 0 OrElse grade3 < 0 OrElse _
       grade1 > 100 OrElse grade2 > 100 OrElse grade3 > 100 Then
        lblStatus.Text = "Each grade must be from 0 to 100 only."
        lblRemarks1.Text = "Out of range"
        lblRemarks2.Text = "Out of range"
        lblRemarks3.Text = "Out of range"
        lblAverage.Text = "0.00"
        Return False
    End If

    Return True
End Function

Private Sub btnEvaluate_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
btnEvaluate.Click
    If Not ReadGrades() Then Return

    ' Average of the three grades (Double so decimals are kept)
    averageGrade = (grade1 + grade2 + grade3) / 3
    lblAverage.Text = averageGrade.ToString("0.00")

    ' Remarks 1 : passed / conditional / failed
    Select Case averageGrade
        Case Is >= 75
            lblRemarks1.Text = "Passed!"
        Case Is >= 70
            lblRemarks1.Text = "Conditional!"
        Case Else
            lblRemarks1.Text = "Failed!"
    End Select

    ' Remarks 2 : letter grade
    Select Case averageGrade
        Case Is >= 95
            lblRemarks2.Text = "A"
        Case Is >= 89
            lblRemarks2.Text = "B"
        Case Is >= 80
            lblRemarks2.Text = "C"
        Case Is >= 75
            lblRemarks2.Text = "D"
        Case Is >= 70
            lblRemarks2.Text = "E"
        Case Else
            lblRemarks2.Text = "F"
    End Select

    ' Remarks 3 : numeric rating
    Select Case averageGrade
        Case Is >= 98
```

```
                lblRemarks3.Text = "1.00"
            Case Is >= 95
                lblRemarks3.Text = "1.25"
            Case Is >= 92
                lblRemarks3.Text = "1.50"
            Case Is >= 89
                lblRemarks3.Text = "1.75"
            Case Is >= 86
                lblRemarks3.Text = "2.00"
            Case Is >= 83
                lblRemarks3.Text = "2.25"
            Case Is >= 80
                lblRemarks3.Text = "2.50"
            Case Is >= 77
                lblRemarks3.Text = "2.75"
            Case Is >= 75
                lblRemarks3.Text = "3.00"
            Case Is >= 70
                lblRemarks3.Text = "4.00"
            Case Else
                lblRemarks3.Text = "5.00"
        End Select

        ShowBadgeColor()
        lblStatus.Text = "Average " & averageGrade.ToString("0.00") & " evaluated with three Select Case
    blocks."
    End Sub

    Private Sub ShowBadgeColor()
        If averageGrade >= 75 Then
            pnlAveBadge.BackColor = System.Drawing.Color.FromArgb(6, 78, 59)
            lblRemarks1.ForeColor = System.Drawing.Color.FromArgb(22, 163, 74)
        ElseIf averageGrade >= 70 Then
            pnlAveBadge.BackColor = System.Drawing.Color.FromArgb(120, 53, 15)
            lblRemarks1.ForeColor = System.Drawing.Color.FromArgb(217, 119, 6)
        Else
            pnlAveBadge.BackColor = System.Drawing.Color.FromArgb(127, 29, 29)
            lblRemarks1.ForeColor = System.Drawing.Color.FromArgb(220, 38, 38)
        End If
    End Sub

    Private Sub btnClear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnClear.Click
        ResetScreen()
    End Sub

    Private Sub btnExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnExit.Click
        Application.Exit()
    End Sub
End Class
```

Activity 7

Activity 7 - Integrative Program 1 - Interest, Square and Class Average

Project : Act7_IntegProg1_SW20260720 | Author : Mangalili, Fhetz | BSIT 3B

Description

Activity 7 is a multi-tab integrative exercise. One tab computes compound interest with a For loop and a NumericUpDown for the number of years, one tab draws a square out of a fill character using nested For loops, one tab collects grades in a ListBox and counts the letter grade distribution with Select Case, and one tab converts the numbers 1 to 255 into octal, hexadecimal and binary.

Objectives

- Use For loops, Do While loops and NumericUpDown values.
- Apply Select Case for tallying grades.
- Use Convert.ToString(number, base) for number system conversion.

Source Code

Form1.vb

```
 Public Class Form1

    Private Sub calculateButton_Click(ByVal sender As Object,
            ByVal e As EventArgs) Handles calculateButton.Click

        Dim principal As Decimal = Val(principalTextBox.Text)
        Dim rate As Decimal = Val(interestRateTextBox.Text)
        Dim amount As Decimal

        resultsListBox.Items.Add(
            "Year" & vbTab & "Amount on Deposit")

        For yearCounter As Integer = 1 To yearsNumericUpDown.Value
            amount = principal * ((1 + rate / 100) ^ yearCounter)
            resultsListBox.Items.Add(yearCounter & vbTab &
                String.Format("{0:C}", amount))
        Next
    End Sub

    Private Sub principalTextBox_TextChanged(ByVal sender As Object,
            ByVal e As EventArgs) Handles principalTextBox.TextChanged
        resultsListBox.Items.Clear()
    End Sub

    Private Sub interestRateTextBox_TextChanged(ByVal sender As Object,
            ByVal e As EventArgs) Handles interestRateTextBox.TextChanged
        resultsListBox.Items.Clear()
    End Sub

    Private Sub yearsNumericUpDown_ValueChanged(ByVal sender As Object,
            ByVal e As EventArgs) Handles yearsNumericUpDown.ValueChanged
        resultsListBox.Items.Clear()
    End Sub

    Private Sub displaySquareButton_Click(ByVal sender As Object,
            ByVal e As EventArgs) Handles displaySquareButton.Click

        outputTextBox.Clear()

        Dim fillCharacter As String = fillCharacterTextBox.Text
        Dim sideLength As Integer = sideLengthNumericUpDown.Value

        For row As Integer = 1 To sideLength
            For column As Integer = 1 To sideLength
                outputTextBox.AppendText(fillCharacter & " ")
            Next column

            outputTextBox.AppendText(vbCrLf)
        Next row
    End Sub

    Private Sub fillCharacterTextBox_TextChanged(ByVal sender As Object,
            ByVal e As EventArgs) Handles fillCharacterTextBox.TextChanged
        outputTextBox.Clear()
```

```
End Sub

Private Sub sideLengthNumericUpDown_ValueChanged(ByVal sender As Object,
        ByVal e As EventArgs) Handles sideLengthNumericUpDown.ValueChanged
    outputTextBox.Clear()
End Sub

Dim aCount As Integer
Dim bCount As Integer
Dim cCount As Integer
Dim dCount As Integer
Dim fCount As Integer
Dim perfectScoreCount As Integer

Private Sub submitGradeButton_Click(ByVal sender As Object,
        ByVal e As EventArgs) Handles submitGradeButton.Click

    Dim grade As Integer = Val(gradeTextBox.Text)

    gradesListBox.Items.Add(grade)
    gradeTextBox.Clear()

    Select Case grade
        Case 100
            perfectScoreCount += 1
            aCount += 1
        Case 90 To 99
            aCount += 1
        Case 80 To 89
            bCount += 1
        Case 70 To 79
            cCount += 1
        Case 60 To 69
            dCount += 1
        Case Else
            fCount += 1
    End Select

    gradeTextBox.Focus()
End Sub

Private Sub calculateAverageButton_Click(ByVal sender As Object,
        ByVal e As EventArgs) Handles calculateAverageButton.Click

    Dim total As Integer
    Dim gradeCounter As Integer
    Dim grade As Integer
    Dim average As Double

    total = 0
    gradeCounter = 0

    Do While gradeCounter < gradesListBox.Items.Count
        grade = gradesListBox.Items(gradeCounter)
        total += grade
        gradeCounter += 1
    Loop

    If gradeCounter <> 0 Then
        average = total / gradeCounter

        classAverageLabel.Text = "Total of the " & gradeCounter &
            " grade(s) is " & total & vbCrLf & "Class average is " &
            String.Format("{0:F}", average) & vbCrLf & vbCrLf

        classAverageLabel.Text &=
            "Letter grade summary:" & vbCrLf & "A: " & aCount &
            vbCrLf & "B: " & bCount & vbCrLf & "C: " & cCount &
            vbCrLf & "D: " & dCount & vbCrLf & "F: " & fCount &
            vbCrLf & "Perfect scores: " & perfectScoreCount
    Else
        classAverageLabel.Text = "No grades were entered"
    End If
End Sub

Private Sub clearGradesButton_Click(ByVal sender As Object,
        ByVal e As EventArgs) Handles clearGradesButton.Click

    gradesListBox.Items.Clear()
    classAverageLabel.Text = String.Empty
End Sub

Private Sub Form1_Load(ByVal sender As Object,
        ByVal e As EventArgs) Handles MyBase.Load

    decimalOutputTextBox.AppendText("Decimal" & vbTab & "Octal" &
        vbTab & "Hexadecimal" & vbTab & "Binary" & vbCrLf)
```

```
        For number As Integer = 1 To 255
            decimalOutputTextBox.AppendText(number & vbTab &
                Convert.ToString(number, 8) & vbTab &
                Convert.ToString(number, 16).ToUpper() & vbTab & vbTab &
                Convert.ToString(number, 2) & vbCrLf)
        Next number
    End Sub

    Private Sub decimalOutputTextBox_TextChanged(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles decimalOutputTextBox.TextChanged

    End Sub

    Private Sub tabClassAverage_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    tabClassAverage.Click

    End Sub

    Private Sub TabPage1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    TabPage1.Click
        'square, rectangle, trapesoid, triangle, circle
    End Sub

    Private Sub TabPage2_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    TabPage2.Click

    End Sub
End Class
```

Activity 8

Activity 8 - The Message (MessageBox Options)

Project : Act8_SW20260805 | Author : Mangalili, Fhetz | BSIT 3B

Description

The Message activity collects everything that a MessageBox needs: a title, a message body, one icon, one button set and one default button. When Display is clicked the program validates the five groups and then shows the message box with the selected options. The answer of the user is reported back in another message box.

The form was redesigned for this compilation from four overlapping GroupBoxes into a two-column dashboard: a compose card on the left and three option cards on the right. The code was rewritten with small helper functions - ValidateEntry, ChosenIcon, ChosenButtons, ChosenDefaultButton and DescribeResult - so that the long chain of If statements in the original version is no longer needed.

Objectives

- Use MessageBox.Show with a title, message, buttons, icon and default button.
- Read DialogResult values and describe the answer of the user.
- Validate user entries before showing a dialog.

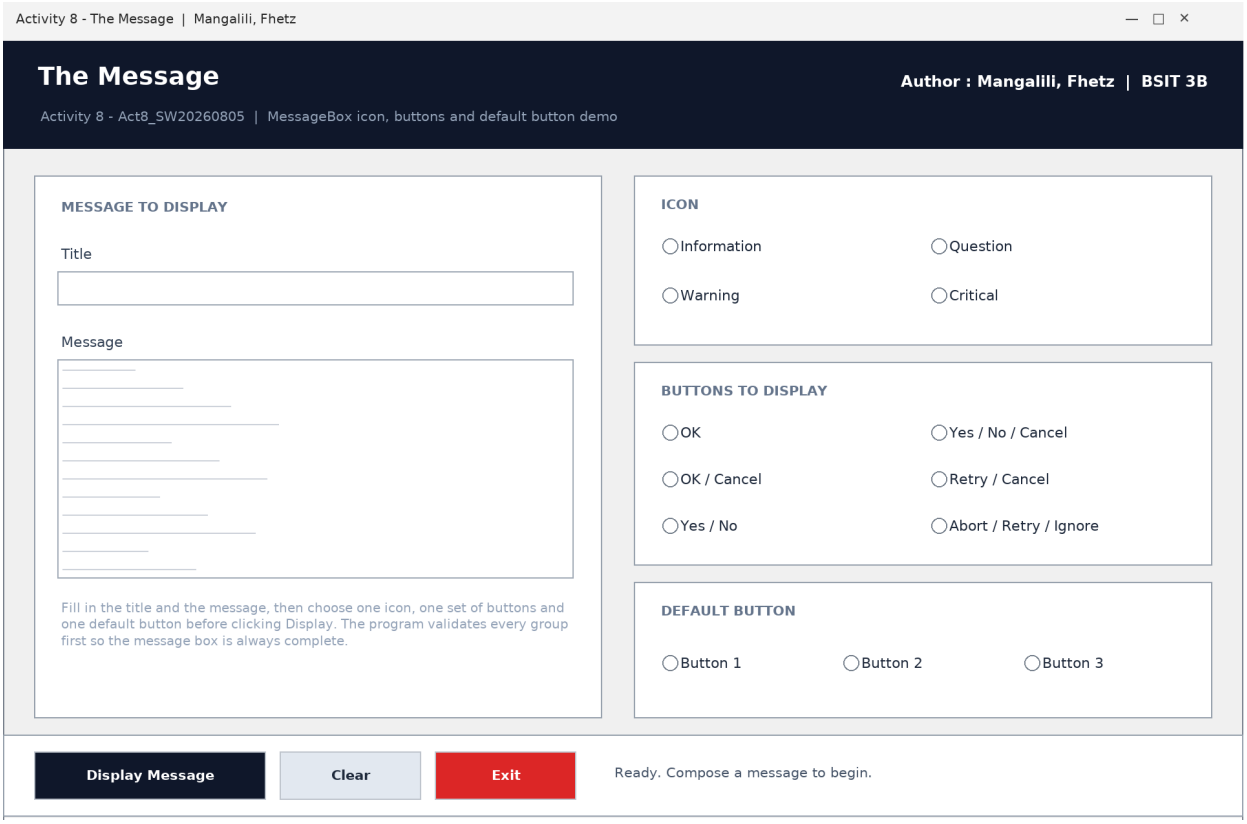


Figure 8.1 - Form design of Activity 8 - The Message

Source Code

Form1.vb

```
' ============================================================
' Activity 8 - Act8_SW20260805
' The Message : composes a MessageBox with a title, a message,
' an icon, a button set and a default button.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
Public Class Form1

    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
```

```
    PrepareForm()
End Sub

Private Sub PrepareForm()
    txtTitle.Clear()
    txtMessage.Clear()
    ClearRadioButtons(pnlIcons)
    ClearRadioButtons(pnlButtons)
    ClearRadioButtons(pnlDefault)
    lblStatus.Text = "Ready. Compose a message to begin."
    txtTitle.Focus()
End Sub

Private Sub ClearRadioButtons(ByVal host As Control)
    For Each child As Control In host.Controls
        If TypeOf child Is RadioButton Then
            CType(child, RadioButton).Checked = False
        End If
    Next
End Sub

' ------------------------- validation -------------------------
Private Function ValidateEntry() As Boolean
    If txtTitle.Text.Trim() = "" Then
        MessageBox.Show("Please enter a title.", "Missing Title", _
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        txtTitle.Focus()
        Return False
    End If

    If txtMessage.Text.Trim() = "" Then
        MessageBox.Show("Please enter a message.", "Missing Message", _
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        txtMessage.Focus()
        Return False
    End If

    If Not (rbInfo.Checked OrElse rbWarning.Checked OrElse rbQuestion.Checked OrElse rbCritical.Checked)
 Then
        MessageBox.Show("Please select one icon.", "Icon Required", _
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Return False
    End If

    If Not (rbOk.Checked OrElse rbOkCancel.Checked OrElse rbYesNo.Checked OrElse _
            rbYesNoCancel.Checked OrElse rbRetryCancel.Checked OrElse rbAbortRetryIgnore.Checked) Then
        MessageBox.Show("Please select the buttons to display.", "Buttons Required", _
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Return False
    End If

    If Not (rbDefault1.Checked OrElse rbDefault2.Checked OrElse rbDefault3.Checked) Then
        MessageBox.Show("Please select the default button.", "Default Button Required", _
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Return False
    End If

    Return True
End Function

' --------------------- selected MessageBox parts ---------------------
Private Function ChosenIcon() As MessageBoxIcon
    If rbInfo.Checked Then Return MessageBoxIcon.Information
    If rbWarning.Checked Then Return MessageBoxIcon.Warning
    If rbQuestion.Checked Then Return MessageBoxIcon.Question
    Return MessageBoxIcon.Error
End Function

Private Function ChosenButtons() As MessageBoxButtons
    If rbOk.Checked Then Return MessageBoxButtons.OK
    If rbOkCancel.Checked Then Return MessageBoxButtons.OKCancel
    If rbYesNo.Checked Then Return MessageBoxButtons.YesNo
    If rbYesNoCancel.Checked Then Return MessageBoxButtons.YesNoCancel
    If rbRetryCancel.Checked Then Return MessageBoxButtons.RetryCancel
    Return MessageBoxButtons.AbortRetryIgnore
End Function

Private Function ChosenDefaultButton() As MessageBoxDefaultButton
    If rbDefault2.Checked Then Return MessageBoxDefaultButton.Button2
    If rbDefault3.Checked Then Return MessageBoxDefaultButton.Button3
    Return MessageBoxDefaultButton.Button1
End Function

Private Function DescribeResult(ByVal answer As DialogResult) As String
    Select Case answer
        Case DialogResult.OK
            Return "OK was clicked."
```

```
            Case DialogResult.Cancel
                Return "Cancel was clicked."
            Case DialogResult.Yes
                Return "Yes was clicked."
            Case DialogResult.No
                Return "No was clicked."
            Case DialogResult.Retry
                Return "Retry was clicked."
            Case DialogResult.Abort
                Return "Abort was clicked."
            Case DialogResult.Ignore
                Return "Ignore was clicked."
            Case Else
                Return "The message box was closed without a button answer."
        End Select
    End Function

    Private Sub btnDisplay_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnDisplay.Click
        If Not ValidateEntry() Then Return

        Try
            Dim answer As DialogResult = MessageBox.Show(txtMessage.Text, txtTitle.Text, _
                                                         ChosenButtons(), ChosenIcon(), _
                                                         ChosenDefaultButton())

            lblStatus.Text = DescribeResult(answer)

            MessageBox.Show(DescribeResult(answer), "Result", _
                            MessageBoxButtons.OK, MessageBoxIcon.Information)
        Catch ex As Exception
            MessageBox.Show(ex.Message, "There is an error", _
                            MessageBoxButtons.OK, MessageBoxIcon.Error)
        End Try
    End Sub

    Private Sub btnClear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnClear.Click
        PrepareForm()
    End Sub

    Private Sub btnExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    btnExit.Click
        Application.Exit()
    End Sub
End Class
```

Activity 9

Activity 9 - ListView File Explorer

Project : SW202609301600 | Author : Mangalili, Fhetz | BSIT 3B

Description

This activity introduces the ListView control. The form lists the folders and files of the Documents folder with their type and size, and the user can browse another folder through a FolderBrowserDialog. The Details and List buttons switch the view, and the menu opens the same form again.

Objectives

- Fill a ListView with folders and files using Directory.GetDirectories and Directory.GetFiles.
- Add columns to a ListView and switch between the Details and List views.
- Use FolderBrowserDialog to choose a folder.

Source Code

ListViewSample.vb

```
 Imports System
 Imports System.IO
 Imports System.Diagnostics
 Imports System.Windows.Forms

 Public Class ListViewSample
     Private currentFolder As String = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)

     Public Sub New()
         InitializeComponent()
         AddHandler menuList.Click, AddressOf OpenListForm

         AddHandler menuExit.Click, AddressOf ExitApplication
     End Sub

     Private Sub OpenListForm(ByVal sender As Object, ByVal e As EventArgs)
         Using form As New ListViewSample()
             form.ShowDialog()
         End Using
     End Sub


     Private Sub ExitApplication(ByVal sender As Object, ByVal e As EventArgs)
         Me.Close()
     End Sub

     Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
         LoadFolder(currentFolder)
     End Sub

     Private Sub LoadFolder(ByVal folder As String)
         If Not Directory.Exists(folder) Then Return
         currentFolder = folder
         txtPath.Text = folder
         lv.Items.Clear()
         Try
             For Each directoryName As String In Directory.GetDirectories(folder)
                 Dim item As New ListViewItem(Path.GetFileName(directoryName))
                 item.SubItems.Add("Folder")
                 item.SubItems.Add(Directory.GetLastWriteTime(directoryName).ToString())
                 item.Tag = directoryName
                 lv.Items.Add(item)
             Next
             For Each fileName As String In Directory.GetFiles(folder)
                 Dim info As New FileInfo(fileName)
                 Dim item As New ListViewItem(info.Name)
                 item.SubItems.Add(info.Extension)
                 item.SubItems.Add(info.Length.ToString() & " bytes")
                 item.Tag = fileName
                 lv.Items.Add(item)
             Next
             lbl.Text = lv.Items.Count.ToString() & " item(s)"
         Catch ex As Exception
             lbl.Text = ex.Message
         End Try
```

```
    End Sub

    Private Sub browse_Click(ByVal sender As Object, ByVal e As EventArgs) Handles browse.Click
        Using dialog As New FolderBrowserDialog()
            dialog.SelectedPath = currentFolder
            dialog.Description = "Select a folder"
            If dialog.ShowDialog() = DialogResult.OK Then
                LoadFolder(dialog.SelectedPath)
            End If
        End Using
    End Sub

    Private Sub btnRefresh_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRefresh.Click
        LoadFolder(currentFolder)
    End Sub

    Private Sub details_Click(ByVal sender As Object, ByVal e As EventArgs) Handles details.Click
        lv.View = View.Details
    End Sub

    Private Sub list_Click(ByVal sender As Object, ByVal e As EventArgs) Handles list.Click
        lv.View = View.List
    End Sub

    Private Sub lv_DoubleClick(ByVal sender As Object, ByVal e As EventArgs) Handles lv.DoubleClick
        If lv.SelectedItems.Count = 0 Then Return
        Dim selectedPath As String = CStr(lv.SelectedItems(0).Tag)
        If Directory.Exists(selectedPath) Then
            LoadFolder(selectedPath)
        ElseIf File.Exists(selectedPath) Then
            Process.Start(selectedPath)
        End If
    End Sub

    Private Sub lv_ColumnClick(ByVal sender As Object, ByVal e As ColumnClickEventArgs) Handles
    lv.ColumnClick
        lv.ListViewItemSorter = New ListViewComparer(e.Column)
        lv.Sort()
    End Sub

    Private Class ListViewComparer
        Implements System.Collections.IComparer
        Private ReadOnly columnNumber As Integer

        Public Sub New(ByVal column As Integer)
            columnNumber = column
        End Sub

        Public Function Compare(ByVal first As Object, ByVal second As Object) As Integer Implements
    System.Collections.IComparer.Compare
            Dim a As ListViewItem = DirectCast(first, ListViewItem)
            Dim b As ListViewItem = DirectCast(second, ListViewItem)
            Return String.Compare(a.SubItems(columnNumber).Text, b.SubItems(columnNumber).Text, True)
        End Function
    End Class
End Class
```

ListViewSample.Designer.vb

```
Imports System.Windows.Forms
Imports System.Drawing

<Global.Microsoft.VisualBasic.CompilerServices.DesignerGenerated()> _
Partial Class ListViewSample
 Inherits System.Windows.Forms.Form
 Private components As System.ComponentModel.IContainer
  Friend WithEvents txtPath As TextBox
 Friend WithEvents browse As Button
 Friend WithEvents btnRefresh As Button
 Friend WithEvents details As Button
 Friend WithEvents list As Button
 Friend WithEvents lv As ListView
 Friend WithEvents lbl As Label
 Friend WithEvents menuStrip1 As MenuStrip
 Friend WithEvents menuForms As ToolStripMenuItem
 Friend WithEvents menuList As ToolStripMenuItem
 Friend WithEvents menuTree As ToolStripMenuItem
 Friend WithEvents menuOpen As ToolStripMenuItem
 Friend WithEvents menuCombined As ToolStripMenuItem
 Friend WithEvents menuExit As ToolStripMenuItem
 <System.Diagnostics.DebuggerNonUserCode()> Protected Overrides Sub Dispose(ByVal disposing As Boolean)
  If disposing AndAlso components IsNot Nothing Then components.Dispose()
  MyBase.Dispose(disposing)
 End Sub
 <System.Diagnostics.DebuggerStepThrough()> Private Sub InitializeComponent()
  Me.components = New System.ComponentModel.Container()
  Me.txtPath=New TextBox() : Me.browse=New Button() : Me.btnRefresh=New Button() : Me.details=New Button() :
     Me.list=New Button() : Me.lv=New ListView() : Me.lbl=New Label()
```

```
  Me.menuStrip1=New MenuStrip() : Me.menuForms=New ToolStripMenuItem() : Me.menuList=New ToolStripMenuItem()
     : Me.menuTree=New ToolStripMenuItem() : Me.menuOpen=New ToolStripMenuItem() : Me.menuCombined=New
    ToolStripMenuItem() : Me.menuExit=New ToolStripMenuItem()
  Me.SuspendLayout()
  Me.menuForms.Text="Forms"
  Me.menuList.Text="ListView File Explorer"
  Me.menuTree.Text="TreeView File Explorer"
  Me.menuOpen.Text="OpenFileDialog File Explorer"
  Me.menuCombined.Text="Combined File Explorer"
  Me.menuExit.Text="Exit"
  Me.menuForms.DropDownItems.AddRange(New ToolStripItem()
    {Me.menuList,Me.menuTree,Me.menuOpen,Me.menuCombined,Me.menuExit})
  Me.menuStrip1.Items.Add(Me.menuForms)
  Me.menuStrip1.Dock=DockStyle.Top
  Me.txtPath.SetBounds(10,38,560,25) : Me.browse.SetBounds(580,36,85,28) : Me.browse.Text="Browse..." :
    Me.btnRefresh.Se ...
' [very long data line omitted in this document]
  Me.ClientSize = New System.Drawing.Size(900, 560)
  Me.Text = "Question 6 - ListView File Explorer"
  Me.StartPosition = FormStartPosition.CenterScreen
  Me.Controls.Add(Me.lv)
  Me.ResumeLayout(False)
 End Sub
End Class
```

Activity 10

Activity 10 - Panel and SplitContainer Laboratory

Project : SW20261002 - PanelSplitPanelDemo | Author : Mangalili, Fhetz | BSIT 3B

Description

Activity 10 is a laboratory for two container controls. Form1 shows the members of the Panel control - Visible, BackColor, BorderStyle, BringToFront, SendToBack, Contains, Focus, Refresh, Invalidate and Dispose - while Form2 shows two nested SplitContainer controls with collapsing, orientation switching and splitter locking.

Both forms and both code files were completely rebuilt for this compilation. Instead of the old plain buttons that only changed a status label, Form1 now has a navigation rail, a workspace with a parent panel and two child panels, an API inspector and an event log that records every action with a counter. Form2 was replaced by a Split Container Lab with a category list, a record list, a details pane, a reset layout button and a ratio based SetRatioDistance helper that keeps the splitter inside the allowed minimum sizes.

Objectives

- Demonstrate the properties and methods of the Panel control.
- Use a vertical and a horizontal SplitContainer, including nested panels.
- Collapse, expand, re-orient and lock splitters at run time.

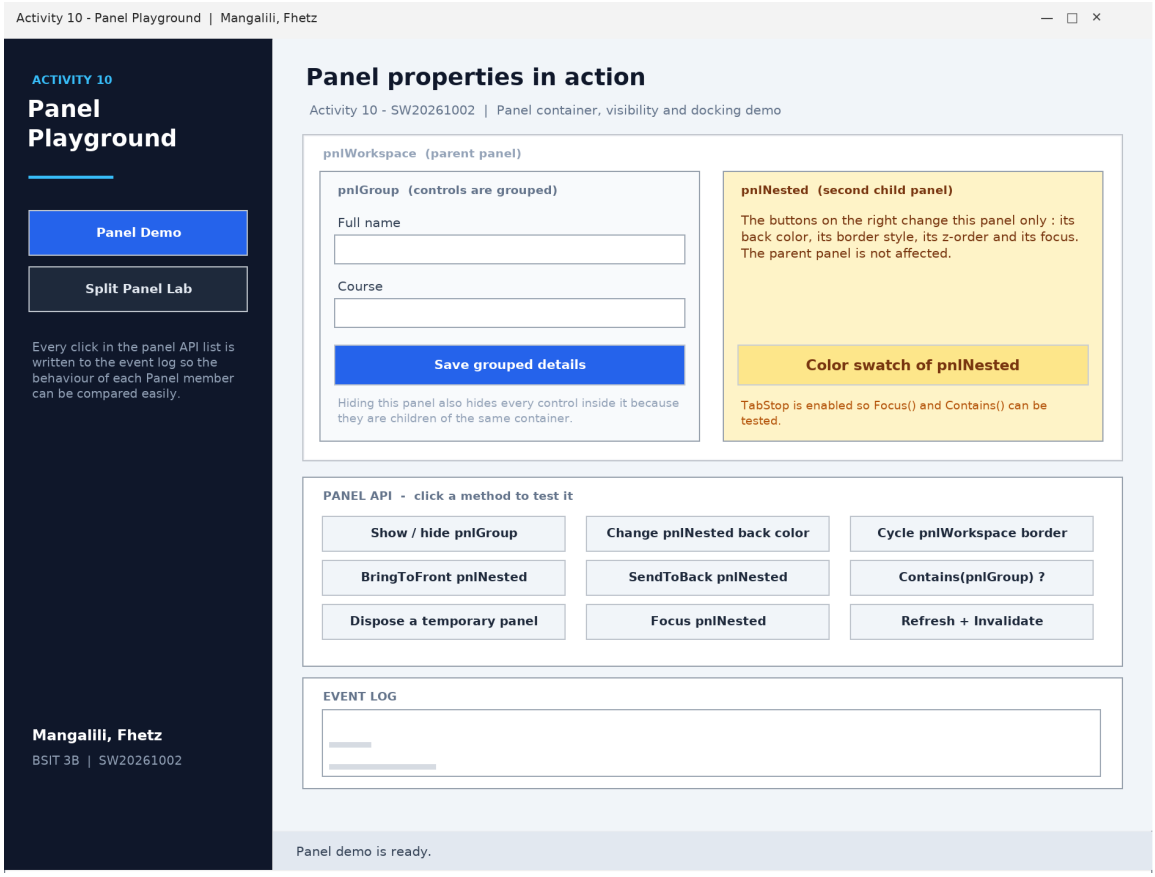


Figure 10.1 - Form design of Activity 10 - Panel Playground

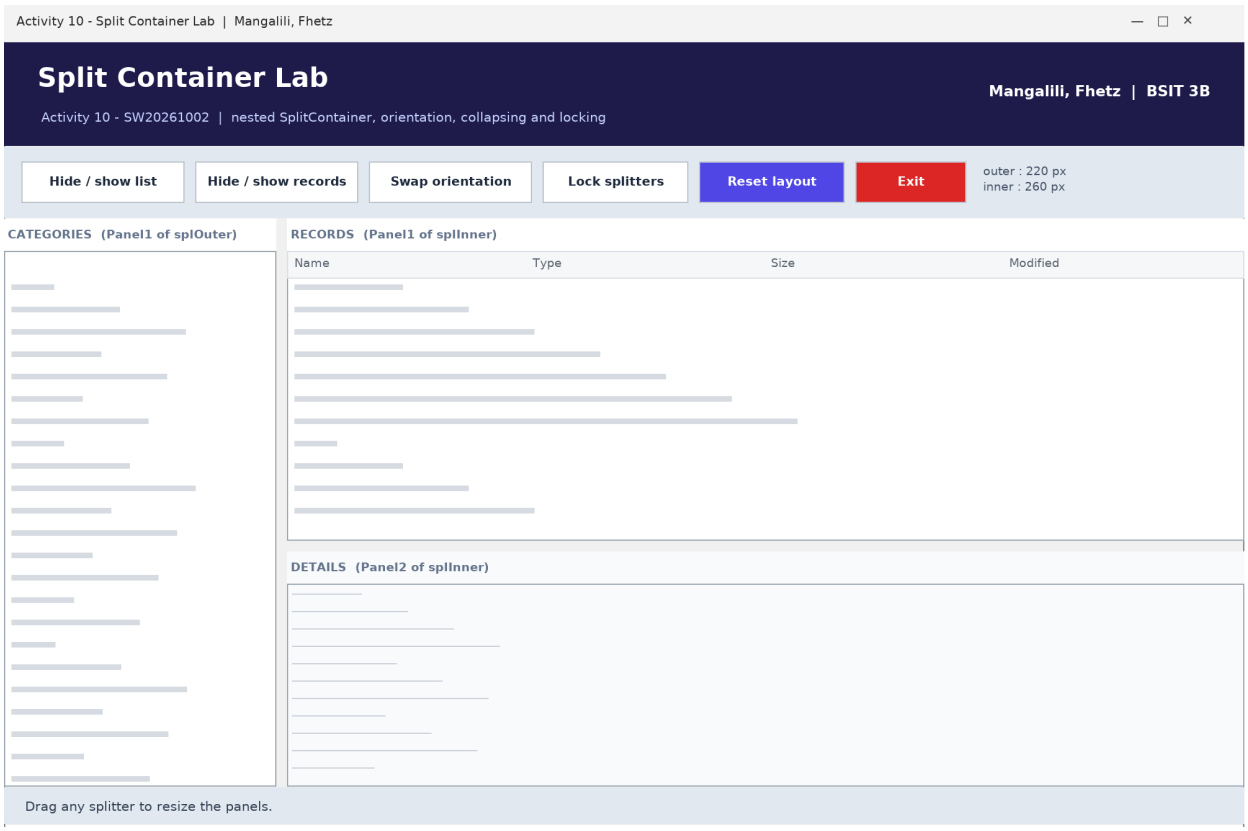


Figure 10.2 - Form design of Activity 10 - Split Container Lab

Source Code

Form1.vb (Panel Playground)

```
' ============================================================
' Activity 10 - SW20261002  (Panel Demo)
' Demonstrates the most common members of the Panel control :
' Visible, BackColor, BorderStyle, BringToFront, SendToBack,
' Contains, Focus, Refresh, Invalidate and Dispose.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
Option Strict On
Option Explicit On

Public Class Form1

    Private actionCounter As Integer = 0
    Private colorIndex As Integer = 0
    Private borderIndex As Integer = 0

    Private ReadOnly palette As Color() = { _
        Color.FromArgb(254, 243, 199), _
        Color.FromArgb(219, 234, 254), _
        Color.FromArgb(220, 252, 231)}

    Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        WriteLog("Form1 loaded - Panel Playground is ready.")
        WriteLog("pnlWorkspace contains " & pnlWorkspace.Controls.Count.ToString() & " child control(s).")
        pnlNested.TabStop = True
        lblStatus.Text = "Panel demo is ready."
    End Sub

    ' ------------------------------------------------------------
    ' helper routines
    ' ------------------------------------------------------------
    Private Sub WriteLog(ByVal message As String)
        actionCounter += 1
        lstLog.Items.Insert(0, actionCounter.ToString("00") & "  " & message)
        lblStatus.Text = message
        lblCounter.Text = actionCounter.ToString() & " action(s) performed"
    End Sub

    Private Sub OpenSplitLab()
        Dim lab As New Form2()
        lab.Owner = Me
        AddHandler lab.FormClosed, AddressOf SplitLabClosed
        lab.Show()
```

```
    Me.Hide()
End Sub

Private Sub SplitLabClosed(ByVal sender As Object, ByVal e As FormClosedEventArgs)
    If Not Me.IsDisposed Then
        Me.Show()
        WriteLog("Split Panel Lab was closed, Panel Playground is shown again.")
    End If
End Sub

' ------------------------------------------------------------
' navigation
' ------------------------------------------------------------
Private Sub btnNavSplit_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnNavSplit.Click
    OpenSplitLab()
End Sub

Private Sub mnuSplitLab_Click(ByVal sender As Object, ByVal e As EventArgs) Handles mnuSplitLab.Click
    OpenSplitLab()
End Sub

Private Sub btnNavPanel_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnNavPanel.Click
    WriteLog("You are already on the Panel demo.")
End Sub

Private Sub mnuExit_Click(ByVal sender As Object, ByVal e As EventArgs) Handles mnuExit.Click
    Application.Exit()
End Sub

' ------------------------------------------------------------
' grouped controls demo
' ------------------------------------------------------------
Private Sub btnSave_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnSave.Click
    Dim fullName As String = txtName.Text.Trim()
    Dim course As String = txtCourse.Text.Trim()

    If fullName = "" Then
        WriteLog("Nothing saved - the full name box is empty.")
        txtName.Focus()
        Return
    End If

    If course = "" Then
        course = "no course given"
    End If

    WriteLog("Saved from pnlGroup : " & fullName & " - " & course)
End Sub

' ------------------------------------------------------------
' Panel API demo buttons
' ------------------------------------------------------------
Private Sub btnToggleGroup_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
btnToggleGroup.Click
    pnlGroup.Visible = Not pnlGroup.Visible
    btnToggleGroup.Text = If(pnlGroup.Visible, "Show / hide pnlGroup", "pnlGroup is hidden - click to
show")
    WriteLog("pnlGroup.Visible = " & pnlGroup.Visible.ToString())
End Sub

Private Sub btnSwapColor_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnSwapColor.Click
    colorIndex += 1
    If colorIndex > palette.Length - 1 Then colorIndex = 0

    pnlNested.BackColor = palette(colorIndex)
    lblNestedBadge.BackColor = ControlPaint.Light(palette(colorIndex))
    WriteLog("pnlNested.BackColor changed to " & ColorName(palette(colorIndex)) & ".")
End Sub

Private Function ColorName(ByVal value As Color) As String
    If value = palette(0) Then Return "amber"
    If value = palette(1) Then Return "blue"
    Return "green"
End Function

Private Sub btnCycleBorder_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
btnCycleBorder.Click
    borderIndex += 1
    Select Case borderIndex Mod 3
        Case 0
            pnlWorkspace.BorderStyle = BorderStyle.Fixed3D
        Case 1
            pnlWorkspace.BorderStyle = BorderStyle.FixedSingle
        Case 2
            pnlWorkspace.BorderStyle = BorderStyle.None
    End Select
    WriteLog("pnlWorkspace.BorderStyle = " & pnlWorkspace.BorderStyle.ToString())
```

```
    End Sub

    Private Sub btnBringFront_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnBringFront.Click
        pnlNested.BringToFront()
        WriteLog("pnlNested.BringToFront() - the child panel is now in front of pnlGroup.")
    End Sub

    Private Sub btnSendBack_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnSendBack.Click
        pnlNested.SendToBack()
        WriteLog("pnlNested.SendToBack() - the child panel moved behind pnlGroup.")
    End Sub

    Private Sub btnContains_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnContains.Click
        Dim hasGroup As Boolean = pnlWorkspace.Contains(pnlGroup)
        Dim hasNested As Boolean = pnlWorkspace.Contains(pnlNested)
        Dim hasOutsider As Boolean = pnlMain.Contains(pnlGroup)
        WriteLog("Contains -> pnlGroup : " & hasGroup.ToString() & _
                 ", pnlNested : " & hasNested.ToString() & _
                 ", from pnlMain : " & hasOutsider.ToString())
    End Sub

    Private Sub btnDisposeDemo_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnDisposeDemo.Click
        Dim temporary As New Panel()
        temporary.Controls.Add(New Label())
        temporary.Controls.Add(New TextBox())
        Dim childCount As Integer = temporary.Controls.Count
        temporary.Dispose()
        WriteLog("A temporary panel with " & childCount.ToString() & " child control(s) was disposed
    safely.")
    End Sub

    Private Sub btnFocusPanel_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnFocusPanel.Click
        Dim gotFocus As Boolean = pnlNested.Focus()
        WriteLog("pnlNested.Focus() returned " & gotFocus.ToString() & ".")
    End Sub

    Private Sub btnRedraw_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRedraw.Click
        pnlNested.Refresh()
        pnlWorkspace.Invalidate(True)
        WriteLog("Refresh() and Invalidate(True) requested a redraw of the child panels.")
    End Sub
End Class
```

Form2.vb (Split Container Lab)

```
' ============================================================
' Activity 10 - SW20261002  (SplitContainer Demo)
' Nested SplitContainer practice : two panels in the outer
' splitter, two panels in the inner splitter.
' Author : Mangalili, Fhetz   |   BSIT 3B
' ============================================================
Option Strict On
Option Explicit On

Public Class Form2

    ' one line of sample data
    Private Class RecordItem
        Public Name As String
        Public Category As String
        Public Status As String
        Public Detail As String

        Public Sub New(ByVal recordName As String, ByVal recordCategory As String, _
                       ByVal recordStatus As String, ByVal recordDetail As String)
            Name = recordName
            Category = recordCategory
            Status = recordStatus
            Detail = recordDetail
        End Sub
    End Class

    Private records As New List(Of RecordItem)()
    Private splittersLocked As Boolean = False

    Private Sub Form2_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        BuildSampleData()
        FillCategories()
        ApplyDefaultLayout()

        If lstCategories.Items.Count > 0 Then lstCategories.SelectedIndex = 0

        lblStatus.Text = "Drag any splitter to resize the panels."
        lblCount.Text = records.Count.ToString() & " sample record(s) loaded"
    End Sub
```

```
' ------------------------------------------------------
' sample data
' ------------------------------------------------------
Private Sub BuildSampleData()
    records.Add(New RecordItem("Student Masterlist", "Students", "Active", _
        "The student masterlist holds the complete list of enrolled students of BSIT 3B." & vbCrLf & _
        "Fields : student number, full name, year level, section and status."))
    records.Add(New RecordItem("Grade Sheet", "Students", "Active", _
        "The grade sheet stores the midterm and final grades of every student." & vbCrLf & _
        "Average and remarks are computed automatically."))
    records.Add(New RecordItem("Course Catalog", "Courses", "Active", _
        "The course catalog lists every subject with its code, title and unit count."))
    records.Add(New RecordItem("Class Schedule", "Courses", "Draft", _
        "The class schedule combines a course, a teacher, a day, a time and a room."))
    records.Add(New RecordItem("Enrollment Slip", "Enrollments", "Active", _
        "The enrollment slip is printed for each student after the subjects are confirmed."))
    records.Add(New RecordItem("Subject Load", "Enrollments", "Pending", _
        "The subject load shows the subjects and schedules of one student for the term."))
    records.Add(New RecordItem("Enrollment Report", "Reports", "Active", _
        "A summary of the number of enrolled students per course and per year level."))
    records.Add(New RecordItem("Grade Summary", "Reports", "Draft", _
        "A report that groups the class by remarks : passed, conditional and failed."))
    records.Add(New RecordItem("User Accounts", "Settings", "Active", _
        "User accounts control who can add, edit or delete the records of the system."))
    records.Add(New RecordItem("Backup Preferences", "Settings", "Pending", _
        "Backup preferences decide how often the database file is copied to a safe folder."))
End Sub

Private Sub FillCategories()
    lstCategories.Items.Clear()
    lstCategories.Items.Add("All records")
    lstCategories.Items.Add("Students")
    lstCategories.Items.Add("Courses")
    lstCategories.Items.Add("Enrollments")
    lstCategories.Items.Add("Reports")
    lstCategories.Items.Add("Settings")
End Sub

Private Sub lstCategories_SelectedIndexChanged(ByVal sender As Object, ByVal e As EventArgs) Handles
lstCategories.SelectedIndexChanged
    ShowRecords(CStr(lstCategories.SelectedItem))
End Sub

Private Sub ShowRecords(ByVal category As String)
    lvRecords.BeginUpdate()
    lvRecords.Items.Clear()

    For Each record As RecordItem In records
        If category = "All records" OrElse record.Category = category Then
            Dim row As New ListViewItem(record.Name)
            row.SubItems.Add(record.Category)
            row.SubItems.Add(record.Status)
            row.Tag = record
            lvRecords.Items.Add(row)
        End If
    Next

    lvRecords.EndUpdate()
    txtDetails.Text = "Select a record in the list above to display its details here."
    lblStatus.Text = If(category = "All records", "Showing every record", "Showing the " & category & "
records") & _
                     " - " & lvRecords.Items.Count.ToString() & " item(s)."
    lblCount.Text = "outer splitter : " & splOuter.SplitterDistance.ToString() & " px"
End Sub

Private Sub lvRecords_SelectedIndexChanged(ByVal sender As Object, ByVal e As EventArgs) Handles
lvRecords.SelectedIndexChanged
    If lvRecords.SelectedItems.Count = 0 Then Return

    Dim chosen As RecordItem = CType(lvRecords.SelectedItems(0).Tag, RecordItem)
    txtDetails.Text = chosen.Name & vbCrLf & _
                      "Category : " & chosen.Category & vbCrLf & _
                      "Status : " & chosen.Status & vbCrLf & vbCrLf & _
                      chosen.Detail & vbCrLf & vbCrLf & _
                      "This text is shown inside Panel2 of the inner SplitContainer."
    lblStatus.Text = "Selected : " & chosen.Name
End Sub

' ------------------------------------------------------
' split container layout helpers
' ------------------------------------------------------
Private Sub ApplyDefaultLayout()
    splOuter.Panel1Collapsed = False
    splOuter.Panel2Collapsed = False
    splInner.Panel1Collapsed = False
    splInner.Panel2Collapsed = False
    splInner.Orientation = Orientation.Horizontal
```

```
    SetRatioDistance(splOuter, 0.24)
    SetRatioDistance(splInner, 0.62)
    ShowSplitInfo()
End Sub

' The distance is given as a ratio of the available space so the
' splitter never goes beyond its minimum sizes.
Private Sub SetRatioDistance(ByVal splitter As SplitContainer, ByVal ratio As Double)
    Dim available As Integer
    If splitter.Orientation = Orientation.Vertical Then
        available = splitter.ClientSize.Width
    Else
        available = splitter.ClientSize.Height
    End If

    Dim lowest As Integer = splitter.Panel1MinSize
    Dim highest As Integer = available - splitter.Panel2MinSize - splitter.SplitterWidth
    If highest <= lowest Then Return

    Dim requested As Integer = CInt(available * ratio)
    If requested < lowest Then requested = lowest
    If requested > highest Then requested = highest
    splitter.SplitterDistance = requested
End Sub

Private Sub ShowSplitInfo()
    lblSplitInfo.Text = "outer : " & splOuter.SplitterDistance.ToString() & " px" & vbCrLf & _
                        "inner : " & splInner.SplitterDistance.ToString() & " px"
End Sub

Private Sub btnCollapseLeft_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
btnCollapseLeft.Click
    splOuter.Panel1Collapsed = Not splOuter.Panel1Collapsed
    lblStatus.Text = "splOuter.Panel1Collapsed = " & splOuter.Panel1Collapsed.ToString()
    ShowSplitInfo()
End Sub

Private Sub btnCollapseTop_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
btnCollapseTop.Click
    splInner.Panel1Collapsed = Not splInner.Panel1Collapsed
    lblStatus.Text = "splInner.Panel1Collapsed = " & splInner.Panel1Collapsed.ToString()
    ShowSplitInfo()
End Sub

Private Sub btnSwapOrientation_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
btnSwapOrientation.Click
    Try
        If splInner.Orientation = Orientation.Horizontal Then
            splInner.Orientation = Orientation.Vertical
            lblStatus.Text = "Orientation = Vertical, the inner panels are now left and right."
        Else
            splInner.Orientation = Orientation.Horizontal
            lblStatus.Text = "Orientation = Horizontal, the inner panels are now top and bottom."
        End If
        SetRatioDistance(splInner, 0.62)
        ShowSplitInfo()
    Catch ex As Exception
        lblStatus.Text = "The orientation could not be changed : " & ex.Message
    End Try
End Sub

Private Sub btnLock_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnLock.Click
    splittersLocked = Not splittersLocked
    splOuter.IsSplitterFixed = splittersLocked
    splInner.IsSplitterFixed = splittersLocked
    btnLock.Text = If(splittersLocked, "Unlock splitters", "Lock splitters")
    lblStatus.Text = "IsSplitterFixed = " & splittersLocked.ToString()
End Sub

Private Sub btnReset_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnReset.Click
    splittersLocked = False
    splOuter.IsSplitterFixed = False
    splInner.IsSplitterFixed = False
    btnLock.Text = "Lock splitters"
    ApplyDefaultLayout()
    lblStatus.Text = "The default layout and splitter distances were restored."
End Sub

Private Sub splOuter_SplitterMoved(ByVal sender As Object, ByVal e As SplitterEventArgs) Handles
splOuter.SplitterMoved
    ShowSplitInfo()
    lblStatus.Text = "Outer splitter moved to " & splOuter.SplitterDistance.ToString() & " pixels."
End Sub

Private Sub splInner_SplitterMoved(ByVal sender As Object, ByVal e As SplitterEventArgs) Handles
splInner.SplitterMoved
    ShowSplitInfo()
```

```
        lblStatus.Text = "Inner splitter moved to " & splInner.SplitterDistance.ToString() & " pixels."
    End Sub

    ' ------------------------------------------------------
    ' navigation
    ' ------------------------------------------------------
    Private Sub mnuPanelDemo_Click(ByVal sender As Object, ByVal e As EventArgs) Handles mnuPanelDemo.Click
        Me.Close()
    End Sub

    Private Sub mnuExit_Click(ByVal sender As Object, ByVal e As EventArgs) Handles mnuExit.Click
        Application.Exit()
    End Sub

    Private Sub btnExit_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnExit.Click
        Application.Exit()
    End Sub
End Class
```

Activity 11

Activity 11 - Windows Forms Controls Collection (TabControl, TreeView, ListView, Dialogs)

Project : SW202610021600 | Author : Mangalili, Fhetz | BSIT 3B

Description

The eleventh activity collects the answers to questions 6 to 9 in one TabControl based program: a ListView file explorer, a TreeView file explorer, an OpenFileDialog and SaveFileDialog demonstration, and a combined explorer that uses the TreeView, the ListView and both dialogs together. One tab lists the written answers of the assignment.

Objectives

- Build forms completely in code without the designer.
- Embed one form inside a TabPage of another form.
- Combine TreeView, ListView, OpenFileDialog and SaveFileDialog in one application.

Source Code

Application.vb

```
Imports System
Imports System.Windows.Forms
Module Application
 <STAThread()> Public Sub Main()
  System.Windows.Forms.Application.EnableVisualStyles()
  System.Windows.Forms.Application.SetCompatibleTextRenderingDefault(False)
  System.Windows.Forms.Application.Run(New MainForm())
 End Sub
End Module
```

MainForm.vb

```
Imports System
Imports System.Drawing
Imports System.Windows.Forms

Public Class MainForm
    Inherits Form

    Private tabs As TabControl
    Private explorerTab As TabPage
    Private listTab As TabPage
    Private treeTab As TabPage
    Private answersTab As TabPage

    Public Sub New()
        Me.Text = "SW202610021600 - Windows Forms Controls"
        Me.StartPosition = FormStartPosition.CenterScreen
        Me.WindowState = FormWindowState.Maximized
        Me.MinimumSize = New Size(900, 600)
        tabs = New TabControl()
        tabs.Dock = DockStyle.Fill
        explorerTab = New TabPage("File explorer + dialogs")
        listTab = New TabPage("ListView examples")
        treeTab = New TabPage("TreeView examples")
        answersTab = New TabPage("Assignment answers")
        tabs.TabPages.Add(explorerTab)
        tabs.TabPages.Add(listTab)
        tabs.TabPages.Add(treeTab)
        tabs.TabPages.Add(answersTab)
        Me.Controls.Add(tabs)
        EmbedForm(New CombinedSample(), explorerTab)
        EmbedForm(New ListViewSample(), listTab)
        EmbedForm(New TreeViewSample(), treeTab)
        BuildAnswersTab()
    End Sub

    Private Sub EmbedForm(ByVal child As Form, ByVal page As TabPage)
        child.TopLevel = False
        child.FormBorderStyle = FormBorderStyle.None
        child.Dock = DockStyle.Fill
        page.Controls.Add(child)
        child.Show()
```

```
    End Sub

    Private Sub BuildAnswersTab()
        Dim text As New RichTextBox()
        text.Dock = DockStyle.Fill
        text.ReadOnly = True
        text.Font = New Font("Segoe UI", 11.0F)
        text.Text = "SW202610021600" & Environment.NewLine & Environment.NewLine & _
                    "Question 6 - ListView" & Environment.NewLine & _
                    "The ListView examples tab imitates File Explorer with folder browsing, Details/List
    views, columns, sorting, double-click opening, and a status bar." & Environment.NewLine &
    Environment.NewLine & _
                    "Question 7 - TreeView" & Environment.NewLine & _
                    "The TreeView examples tab displays drives and directories. Nodes are loaded when
    expanded and the selected path is shown." & Environment.NewLine & Environment.NewLine & _
                    "Question 8 - OpenFileDialog" & Environment.NewLine & _
                    "The File explorer + dialogs tab demonstrates OpenFileDialog, file filters, multiple
    selection, CheckFileExists, and RestoreDirectory." & Environment.NewLine & Environment.NewLine & _
                    "Question 9 - Combined form" & Environment.NewLine & _
                    "The File explorer + dialogs tab combines OpenFileDialog, SaveFileDialog, TreeView,
    ListView, folder navigation, and saving a directory listing."
        answersTab.Controls.Add(text)
    End Sub
End Class
```

MenuForm.vb

```
Imports System
Imports System.Drawing
Imports System.Windows.Forms

Public Class MenuForm
    Inherits Form

    Private menuStrip1 As MenuStrip
    Private listMenu As ToolStripMenuItem
    Private treeMenu As ToolStripMenuItem
    Private openMenu As ToolStripMenuItem
    Private combinedMenu As ToolStripMenuItem
    Private exitMenu As ToolStripMenuItem

    Public Sub New()
        Me.Text = "SW202610021600"
        Me.StartPosition = FormStartPosition.CenterScreen
        Me.Size = New Size(720, 400)
        menuStrip1 = New MenuStrip()
        treeMenu = New ToolStripMenuItem("TreeView File Explorer")
        openMenu = New ToolStripMenuItem("OpenFileDialog File Explorer")
        combinedMenu = New ToolStripMenuItem("Combined File Explorer")
        exitMenu = New ToolStripMenuItem("Exit")
        menuStrip1.Items.AddRange(New ToolStripItem() {treeMenu, openMenu, combinedMenu, exitMenu})
        menuStrip1.Dock = DockStyle.Top
        Me.Controls.Add(menuStrip1)
        Me.MainMenuStrip = menuStrip1

        AddHandler treeMenu.Click, AddressOf OpenTree
        AddHandler openMenu.Click, AddressOf OpenDialog
        AddHandler combinedMenu.Click, AddressOf OpenCombined
        AddHandler exitMenu.Click, AddressOf ExitProgram
    End Sub


    Private Sub OpenTree(ByVal sender As Object, ByVal e As EventArgs)
        Using form As New TreeViewSample()
            form.ShowDialog()
        End Using
    End Sub

    Private Sub OpenDialog(ByVal sender As Object, ByVal e As EventArgs)
        Using form As New OpenFileDialogSample()
            form.ShowDialog()
        End Using
    End Sub

    Private Sub OpenCombined(ByVal sender As Object, ByVal e As EventArgs)
        Using form As New CombinedSample()
            form.ShowDialog()
        End Using
    End Sub

    Private Sub ExitProgram(ByVal sender As Object, ByVal e As EventArgs)
        Me.Close()
    End Sub

    Private Sub InitializeComponent()
        Me.SuspendLayout()
        '
        'MenuForm
```

```
        '
        Me.ClientSize = New System.Drawing.Size(729, 387)
        Me.Name = "MenuForm"
        Me.ResumeLayout(False)

    End Sub

    Private Sub MenuForm_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    MyBase.Load

    End Sub
End Class
```

ListViewSample.vb

```
Imports System
Imports System.IO
Imports System.Diagnostics
Imports System.Windows.Forms

Public Class ListViewSample
    Private currentFolder As String = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)

    Public Sub New()
        InitializeComponent()
    End Sub

    Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        LoadFolder(currentFolder)
    End Sub

    Private Sub LoadFolder(ByVal folder As String)
        If Not Directory.Exists(folder) Then Return
        currentFolder = folder
        txtPath.Text = folder
        lv.Items.Clear()
        Try
            For Each directoryName As String In Directory.GetDirectories(folder)
                Dim item As New ListViewItem(Path.GetFileName(directoryName))
                item.SubItems.Add("Folder")
                item.SubItems.Add(Directory.GetLastWriteTime(directoryName).ToString())
                item.Tag = directoryName
                lv.Items.Add(item)
            Next
            For Each fileName As String In Directory.GetFiles(folder)
                Dim info As New FileInfo(fileName)
                Dim item As New ListViewItem(info.Name)
                item.SubItems.Add(info.Extension)
                item.SubItems.Add(info.Length.ToString() & " bytes")
                item.Tag = fileName
                lv.Items.Add(item)
            Next
            lbl.Text = lv.Items.Count.ToString() & " item(s)"
        Catch ex As Exception
            lbl.Text = ex.Message
        End Try
    End Sub

    Private Sub browse_Click(ByVal sender As Object, ByVal e As EventArgs) Handles browse.Click
        Using dialog As New FolderBrowserDialog()
            dialog.SelectedPath = currentFolder
            dialog.Description = "Select a folder"
            If dialog.ShowDialog() = DialogResult.OK Then
                LoadFolder(dialog.SelectedPath)
            End If
        End Using
    End Sub

    Private Sub btnRefresh_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRefresh.Click
        LoadFolder(currentFolder)
    End Sub

    Private Sub details_Click(ByVal sender As Object, ByVal e As EventArgs) Handles details.Click
        lv.View = View.Details
    End Sub

    Private Sub list_Click(ByVal sender As Object, ByVal e As EventArgs) Handles list.Click
        lv.View = View.List
    End Sub

    Private Sub lv_DoubleClick(ByVal sender As Object, ByVal e As EventArgs) Handles lv.DoubleClick
        If lv.SelectedItems.Count = 0 Then Return
        Dim selectedPath As String = CStr(lv.SelectedItems(0).Tag)
        If Directory.Exists(selectedPath) Then
            LoadFolder(selectedPath)
        ElseIf File.Exists(selectedPath) Then
            Process.Start(selectedPath)
        End If
```

```
    End Sub

    Private Sub lv_ColumnClick(ByVal sender As Object, ByVal e As ColumnClickEventArgs) Handles
    lv.ColumnClick
        lv.ListViewItemSorter = New ListViewComparer(e.Column)
        lv.Sort()
    End Sub

    Private Class ListViewComparer
        Implements System.Collections.IComparer
        Private ReadOnly columnNumber As Integer

        Public Sub New(ByVal column As Integer)
            columnNumber = column
        End Sub

        Public Function Compare(ByVal first As Object, ByVal second As Object) As Integer Implements
    System.Collections.IComparer.Compare
            Dim a As ListViewItem = DirectCast(first, ListViewItem)
            Dim b As ListViewItem = DirectCast(second, ListViewItem)
            Return String.Compare(a.SubItems(columnNumber).Text, b.SubItems(columnNumber).Text, True)
        End Function
    End Class
End Class
```

TreeViewSample.vb

```
Imports System
Imports System.IO
Imports System.Windows.Forms

Public Class TreeViewSample
    Public Sub New()
        InitializeComponent()
    End Sub
    Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        BuildTree()
    End Sub
    Private Sub BuildTree()
        tree.Nodes.Clear()
        For Each drive As DriveInfo In DriveInfo.GetDrives()
            If drive.IsReady Then
                Dim node As New TreeNode(drive.Name)
                node.Tag = drive.RootDirectory.FullName
                node.Nodes.Add("Loading...")
                tree.Nodes.Add(node)
            End If
        Next
    End Sub
    Private Sub tree_BeforeExpand(ByVal sender As Object, ByVal e As TreeViewCancelEventArgs) Handles
    tree.BeforeExpand
        Dim folder As String = CStr(e.Node.Tag)
        e.Node.Nodes.Clear()
        Try
            For Each directoryName As String In Directory.GetDirectories(folder)
                Dim child As New TreeNode(Path.GetFileName(directoryName))
                child.Tag = directoryName
                Try
                    If Directory.GetDirectories(directoryName).Length > 0 Then child.Nodes.Add("Loading...")
                Catch
                End Try
                e.Node.Nodes.Add(child)
            Next
        Catch
        End Try
    End Sub
    Private Sub tree_AfterSelect(ByVal sender As Object, ByVal e As TreeViewEventArgs) Handles
    tree.AfterSelect
        selectedPath.Text = CStr(e.Node.Tag)
    End Sub
    Private Sub btnRefresh_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRefresh.Click
        BuildTree()
    End Sub
End Class
```

OpenFileDialogSample.vb

```
Imports System
Imports System.Windows.Forms

Public Class OpenFileDialogSample
    Public Sub New()
        InitializeComponent()
    End Sub
    Private Sub one_Click(ByVal sender As Object, ByVal e As EventArgs) Handles one.Click
        Using dialog As New OpenFileDialog()
            Configure(dialog, False)
            If dialog.ShowDialog() = DialogResult.OK Then ShowFiles(dialog.FileNames)
        End Using
```

```
    End Sub
    Private Sub many_Click(ByVal sender As Object, ByVal e As EventArgs) Handles many.Click
        Using dialog As New OpenFileDialog()
            Configure(dialog, True)
            If dialog.ShowDialog() = DialogResult.OK Then ShowFiles(dialog.FileNames)
        End Using
    End Sub
    Private Sub Configure(ByVal dialog As OpenFileDialog, ByVal allowMultiple As Boolean)
        dialog.Title = "Select file"
        dialog.Filter = "All files (*.*)|*.*|Text files (*.txt)|*.txt|Images (*.jpg;*.png)|*.jpg;*.png"
        dialog.FilterIndex = 1
        dialog.Multiselect = allowMultiple
        dialog.CheckFileExists = True
        dialog.CheckPathExists = True
        dialog.RestoreDirectory = True
    End Sub
    Private Sub ShowFiles(ByVal files() As String)
        selected.Items.Clear()
        For Each fileName As String In files
            selected.Items.Add(fileName)
        Next
    End Sub
    Private Sub clear_Click(ByVal sender As Object, ByVal e As EventArgs) Handles clear.Click
        selected.Items.Clear()
    End Sub
End Class
```

CombinedSample.vb

```
Imports System
Imports System.IO
Imports System.Diagnostics
Imports System.Windows.Forms

Public Class CombinedSample
    Private currentFolder As String = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)
    Public Sub New()
        InitializeComponent()
    End Sub
    Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        BuildTree()
        LoadFolder(currentFolder)
    End Sub
    Private Sub BuildTree()
        tree.Nodes.Clear()
        For Each drive As DriveInfo In DriveInfo.GetDrives()
            If drive.IsReady Then
                Dim node As New TreeNode(drive.Name)
                node.Tag = drive.RootDirectory.FullName
                node.Nodes.Add("Loading...")
                tree.Nodes.Add(node)
            End If
        Next
    End Sub
    Private Sub tree_BeforeExpand(ByVal sender As Object, ByVal e As TreeViewCancelEventArgs) Handles
    tree.BeforeExpand
        Dim folder As String = CStr(e.Node.Tag)
        e.Node.Nodes.Clear()
        Try
            For Each directoryName As String In Directory.GetDirectories(folder)
                Dim child As New TreeNode(Path.GetFileName(directoryName))
                child.Tag = directoryName
                e.Node.Nodes.Add(child)
            Next
        Catch
        End Try
    End Sub
    Private Sub tree_AfterSelect(ByVal sender As Object, ByVal e As TreeViewEventArgs) Handles
    tree.AfterSelect
        currentFolder = CStr(e.Node.Tag)
        LoadFolder(currentFolder)
    End Sub
    Private Sub LoadFolder(ByVal folder As String)
        If Not Directory.Exists(folder) Then Return
        lv.Items.Clear()
        Try
            For Each directoryName As String In Directory.GetDirectories(folder)
                Dim item As New ListViewItem(Path.GetFileName(directoryName))
                item.SubItems.Add("Folder")
                item.Tag = directoryName
                lv.Items.Add(item)
            Next
            For Each fileName As String In Directory.GetFiles(folder)
                Dim item As New ListViewItem(Path.GetFileName(fileName))
                item.SubItems.Add("File")
                item.Tag = fileName
                lv.Items.Add(item)
            Next
```

```
        Catch
        End Try
    End Sub
    Private Sub open_Click(ByVal sender As Object, ByVal e As EventArgs) Handles open.Click
        Using dialog As New OpenFileDialog()
            dialog.Filter = "All files (*.*)|*.*"
            dialog.CheckFileExists = True
            If dialog.ShowDialog() = DialogResult.OK Then Process.Start(dialog.FileName)
        End Using
    End Sub
    Private Sub save_Click(ByVal sender As Object, ByVal e As EventArgs) Handles save.Click
        Using dialog As New SaveFileDialog()
            dialog.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*"
            If dialog.ShowDialog() = DialogResult.OK Then
                Using writer As New StreamWriter(dialog.FileName)
                    For Each item As ListViewItem In lv.Items
                        writer.WriteLine(item.Text & Microsoft.VisualBasic.ControlChars.Tab &
    item.SubItems(1).Text)
                    Next
                End Using
            End If
        End Using
    End Sub
    Private Sub btnRefresh_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRefresh.Click
        BuildTree()
        LoadFolder(currentFolder)
    End Sub
End Class
```

Activity 12

Activity 12 - File Explorer Clone (PanelSplitDemo)

Project : FileExplorerClone - PanelSplitDemo | Author : Mangalili, Fhetz | BSIT 3B

Description

The last activity of the container chapter is a working file explorer. The folder tree is loaded on demand, the file list shows the name, type, size and date modified of every item, and the preview pane shows the text of a file, the picture of an image or the summary of a folder. The layout is built from two SplitContainer controls: one between the tree and the workspace, and one between the file list and the preview.

The whole activity was rewritten for this compilation so that the design and the code are different from the sample. The new name of the program is Mangalili Explorer. The additions are: a Back and Forward history stack, a Home button, an editable address bar with a Go button, a live filter box, column sorting that compares sizes as numbers, hidden files excluded from the list, an image preview that disposes the previous bitmap before showing a new one, a text preview limited to 6000 characters, and buttons that hide the tree or the preview and swap the orientation of the inner splitter.

Objectives

- Combine TreeView, ListView, PictureBox and TextBox inside nested SplitContainers.
- Navigate the file system with Directory, File, FileInfo and DirectoryInfo.
- Preview text, images and folders safely with error handling.

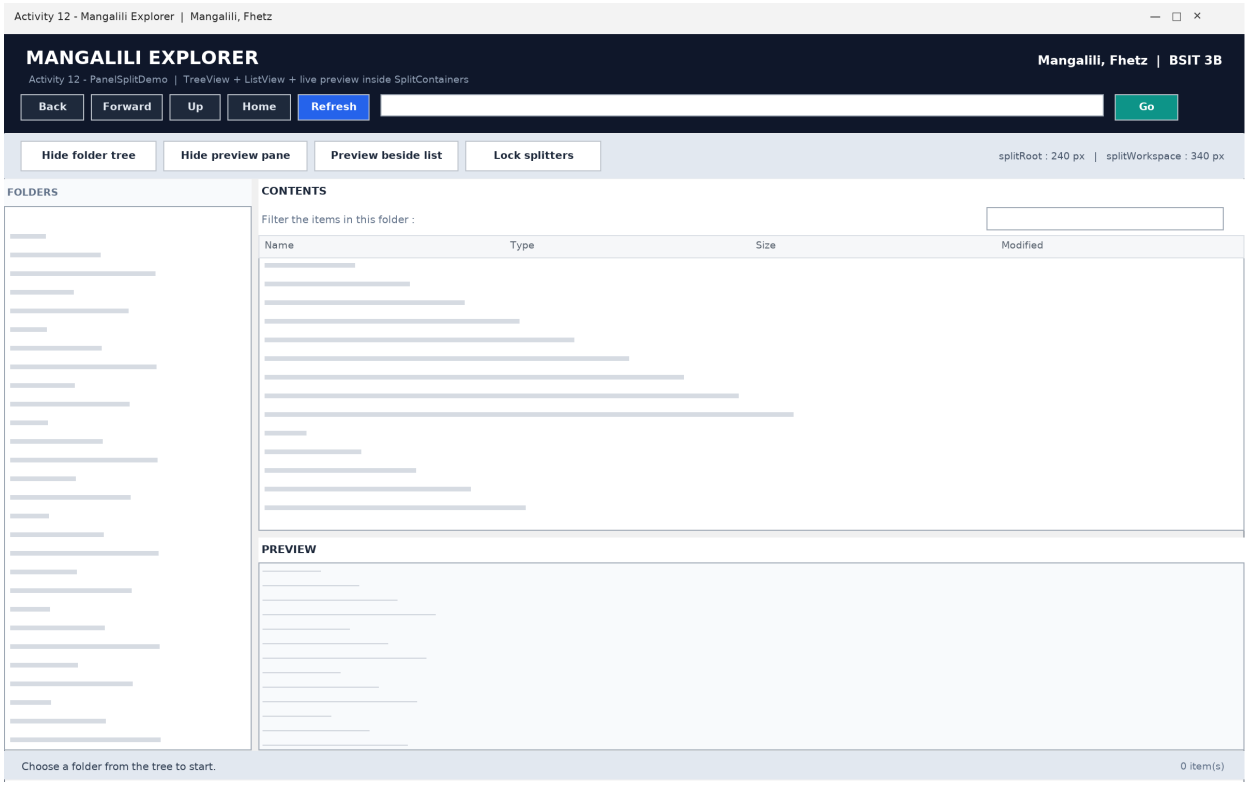


Figure 12.1 - Form design of Activity 12 - Mangalili Explorer

Source Code

Form1.vb

```
' ============================================================
' Activity 12 - PanelSplitDemo  (Mangalili Explorer)
' A small file explorer built with TreeView + ListView + preview
' inside nested SplitContainer controls.
' Author : Mangalili, Fhetz   |   BSIT 3B
```

```
' ============================================================
Option Strict On
Option Explicit On

Imports System.IO
Imports System.Text

Public Class Form1

    Private currentFolder As String = ""
    Private ReadOnly backStack As New Stack(Of String)()
    Private ReadOnly forwardStack As New Stack(Of String)()
    Private syncingTree As Boolean = False
    Private splittersLocked As Boolean = False
    Private lastSortColumn As Integer = -1
    Private sortAscending As Boolean = True

    Private ReadOnly imageTypes As String() = _
        {".jpg", ".jpeg", ".png", ".gif", ".bmp", ".ico", ".tif", ".tiff"}

    Private ReadOnly textTypes As String() = _
        {".txt", ".vb", ".cs", ".xml", ".config", ".csv", ".log", ".md", ".json", _
         ".ini", ".html", ".htm", ".css", ".js", ".sql", ".bat", ".sln", ".vbproj"}

    ' ------------------------------------------------------------
    ' start up
    ' ------------------------------------------------------------
    Private Sub Form1_Load(ByVal sender As Object, ByVal e As EventArgs) Handles MyBase.Load
        Try
            BuildFolderTree()
            ApplyDefaultLayout()
            NavigateTo(StartFolder(), False)
        Catch ex As Exception
            lblStatus.Text = "The explorer could not start : " & ex.Message
        End Try
    End Sub

    Private Function StartFolder() As String
        Dim documents As String = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)
        If Directory.Exists(documents) Then Return documents

        Dim desktopFolder As String = Environment.GetFolderPath(Environment.SpecialFolder.Desktop)
        If Directory.Exists(desktopFolder) Then Return desktopFolder

        Return Environment.CurrentDirectory
    End Function

    ' ------------------------------------------------------------
    ' folder tree
    ' ------------------------------------------------------------
    Private Sub BuildFolderTree()
        tvFolders.BeginUpdate()
        tvFolders.Nodes.Clear()

        AddRootFolder("Desktop", Environment.GetFolderPath(Environment.SpecialFolder.Desktop))
        AddRootFolder("Documents", Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments))
        AddRootFolder("Pictures", Environment.GetFolderPath(Environment.SpecialFolder.MyPictures))
        AddRootFolder("Downloads",
    Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), "Downloads"))

        For Each drive As DriveInfo In DriveInfo.GetDrives()
            If drive.IsReady Then
                AddRootFolder(drive.Name, drive.RootDirectory.FullName)
            End If
        Next

        tvFolders.EndUpdate()
        lblStatus.Text = tvFolders.Nodes.Count.ToString() & " root folder(s) listed."
    End Sub

    Private Sub AddRootFolder(ByVal title As String, ByVal folderPath As String)
        If folderPath = "" OrElse Not Directory.Exists(folderPath) Then Return

        Dim node As New TreeNode(title)
        node.Tag = folderPath
        node.Nodes.Add(New TreeNode("loading"))
        tvFolders.Nodes.Add(node)
    End Sub

    Private Sub tvFolders_BeforeExpand(ByVal sender As Object, ByVal e As TreeViewCancelEventArgs) Handles
    tvFolders.BeforeExpand
        If e.Node.Tag Is Nothing Then Return
        If e.Node.Nodes.Count <> 1 OrElse e.Node.Nodes(0).Text <> "loading" Then Return
        If e.Node.Nodes(0).Tag IsNot Nothing Then Return

        Dim parentPath As String = CStr(e.Node.Tag)
        e.Node.Nodes.Clear()
```

```
    Try
        For Each childPath As String In Directory.GetDirectories(parentPath)
            Dim child As New TreeNode(Path.GetFileName(childPath))
            If child.Text = "" Then child.Text = childPath
            child.Tag = childPath
            child.Nodes.Add(New TreeNode("loading"))
            e.Node.Nodes.Add(child)
        Next
    Catch ex As Exception
        lblStatus.Text = "Some folders cannot be listed : " & ex.Message
    End Try

    If e.Node.Nodes.Count = 0 Then e.Node.Nodes.Add(New TreeNode("empty"))
End Sub

Private Sub tvFolders_AfterSelect(ByVal sender As Object, ByVal e As TreeViewEventArgs) Handles
tvFolders.AfterSelect
    If syncingTree Then Return
    If e.Node.Tag Is Nothing Then Return

    Dim folderPath As String = CStr(e.Node.Tag)
    If Directory.Exists(folderPath) Then
        NavigateTo(folderPath, True)
    End If
End Sub

' ------------------------------------------------------
' navigation
' ------------------------------------------------------
Private Sub NavigateTo(ByVal folderPath As String, ByVal remember As Boolean)
    If folderPath = "" OrElse Not Directory.Exists(folderPath) Then
        lblStatus.Text = "Folder not found : " & folderPath
        Return
    End If

    If remember AndAlso currentFolder <> "" AndAlso currentFolder <> folderPath Then
        backStack.Push(currentFolder)
        forwardStack.Clear()
    End If

    ShowFolder(folderPath)
End Sub

Private Sub ShowFolder(ByVal folderPath As String)
    currentFolder = folderPath
    txtPath.Text = folderPath
    RefreshFileList()
    SyncTreeSelection(folderPath)
    ShowPreview("")
    lblStatus.Text = "Now browsing : " & folderPath
End Sub

Private Sub SyncTreeSelection(ByVal folderPath As String)
    syncingTree = True
    Try
        Dim node As TreeNode = FindNode(tvFolders.Nodes, folderPath)
        If node IsNot Nothing Then
            tvFolders.SelectedNode = node
            node.EnsureVisible()
        End If
    Finally
        syncingTree = False
    End Try
End Sub

Private Function FindNode(ByVal siblings As TreeNodeCollection, ByVal folderPath As String) As TreeNode
    For Each node As TreeNode In siblings
        If node.Tag IsNot Nothing AndAlso _
           String.Equals(CStr(node.Tag), folderPath, StringComparison.OrdinalIgnoreCase) Then
            Return node
        End If

        If node.IsExpanded AndAlso node.Nodes.Count > 0 Then
            Dim found As TreeNode = FindNode(node.Nodes, folderPath)
            If found IsNot Nothing Then Return found
        End If
    Next
    Return Nothing
End Function

Private Sub btnBack_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnBack.Click
    If backStack.Count = 0 Then
        lblStatus.Text = "There is no previous folder in the history."
        Return
    End If

    forwardStack.Push(currentFolder)
```

```
    ShowFolder(backStack.Pop())
End Sub

Private Sub btnForward_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnForward.Click
    If forwardStack.Count = 0 Then
        lblStatus.Text = "There is no next folder in the history."
        Return
    End If

    backStack.Push(currentFolder)
    ShowFolder(forwardStack.Pop())
End Sub

Private Sub btnUp_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnUp.Click
    If currentFolder = "" Then Return

    Dim parent As String = Path.GetDirectoryName(currentFolder)
    If parent Is Nothing OrElse parent = "" Then
        lblStatus.Text = "This is already the top of the drive."
        Return
    End If

    NavigateTo(parent, True)
End Sub

Private Sub btnHome_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnHome.Click
    NavigateTo(StartFolder(), True)
End Sub

Private Sub btnRefresh_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnRefresh.Click
    RefreshFileList()
    lblStatus.Text = "Folder refreshed."
End Sub

Private Sub btnGo_Click(ByVal sender As Object, ByVal e As EventArgs) Handles btnGo.Click
    NavigateTo(txtPath.Text.Trim(), True)
End Sub

Private Sub txtPath_KeyDown(ByVal sender As Object, ByVal e As KeyEventArgs) Handles txtPath.KeyDown
    If e.KeyCode = Keys.Enter Then
        e.SuppressKeyPress = True
        NavigateTo(txtPath.Text.Trim(), True)
    End If
End Sub

' ---------------------------------------------------------
' file list
' ---------------------------------------------------------
Private Sub RefreshFileList()
    lvFiles.BeginUpdate()
    lvFiles.Items.Clear()

    If currentFolder = "" OrElse Not Directory.Exists(currentFolder) Then
        lvFiles.EndUpdate()
        lblCount.Text = "0 item(s)"
        Return
    End If

    Dim keyword As String = txtFilter.Text.Trim()

    Try
        For Each folderPath As String In Directory.GetDirectories(currentFolder)
            Dim folderInfo As New DirectoryInfo(folderPath)
            If IsHidden(folderInfo) Then Continue For
            If Not Matches(keyword, folderInfo.Name) Then Continue For
            AddRow(folderInfo.Name, "File folder", "", folderInfo.LastWriteTime, folderInfo.FullName)
        Next

        For Each filePath As String In Directory.GetFiles(currentFolder)
            Dim fileInfo As New FileInfo(filePath)
            If IsHidden(fileInfo) Then Continue For
            If Not Matches(keyword, fileInfo.Name) Then Continue For
            AddRow(fileInfo.Name, TypeLabel(fileInfo), SizeLabel(fileInfo.Length), _
                   fileInfo.LastWriteTime, fileInfo.FullName)
        Next
    Catch ex As Exception
        lblStatus.Text = "This folder cannot be fully listed : " & ex.Message
    Finally
        lvFiles.EndUpdate()
    End Try

    If lastSortColumn >= 0 Then ApplySort(lastSortColumn, False)

    lblCount.Text = lvFiles.Items.Count.ToString() & " item(s)   |   " & currentFolder
End Sub

Private Sub AddRow(ByVal name As String, ByVal kind As String, ByVal size As String, _
```

```
                  ByVal modified As Date, ByVal fullPath As String)
    Dim row As New ListViewItem(name)
    row.SubItems.Add(kind)
    row.SubItems.Add(size)
    row.SubItems.Add(modified.ToString("MMM d, yyyy  h:mm tt"))
    row.Tag = fullPath
    lvFiles.Items.Add(row)
End Sub

Private Function IsHidden(ByVal info As FileSystemInfo) As Boolean
    Return (info.Attributes And FileAttributes.Hidden) = FileAttributes.Hidden
End Function

Private Function Matches(ByVal keyword As String, ByVal itemName As String) As Boolean
    If keyword = "" Then Return True
    Return itemName.IndexOf(keyword, StringComparison.OrdinalIgnoreCase) >= 0
End Function

Private Function TypeLabel(ByVal info As FileInfo) As String
    If info.Extension = "" Then Return "File"
    Return info.Extension.Substring(1).ToUpperInvariant() & " file"
End Function

Private Function SizeLabel(ByVal bytes As Long) As String
    If bytes >= 1073741824 Then Return (bytes / 1073741824.0).ToString("0.00") & " GB"
    If bytes >= 1048576 Then Return (bytes / 1048576.0).ToString("0.00") & " MB"
    If bytes >= 1024 Then Return (bytes / 1024.0).ToString("0.0") & " KB"
    Return bytes.ToString() & " B"
End Function

Private Sub txtFilter_TextChanged(ByVal sender As Object, ByVal e As EventArgs) Handles
txtFilter.TextChanged
    If currentFolder = "" Then Return
    RefreshFileList()
End Sub

' ------------------------------------------------------
' sorting
' ------------------------------------------------------
Private Sub lvFiles_ColumnClick(ByVal sender As Object, ByVal e As ColumnClickEventArgs) Handles
lvFiles.ColumnClick
    If lastSortColumn = e.Column Then
        sortAscending = Not sortAscending
    Else
        lastSortColumn = e.Column
        sortAscending = True
    End If

    ApplySort(e.Column, sortAscending)
    lblStatus.Text = "Sorted by column " & (e.Column + 1).ToString() & If(sortAscending, " (A to Z)", "
(Z to A)")
End Sub

Private Sub ApplySort(ByVal column As Integer, ByVal ascending As Boolean)
    lvFiles.ListViewItemSorter = New ItemSorter(column, ascending)
    lvFiles.Sort()
End Sub

' Compares two ListView rows; the size column is compared as a number.
Friend Class ItemSorter
    Implements System.Collections.IComparer

    Private ReadOnly columnIndex As Integer
    Private ReadOnly ascendingOrder As Boolean

    Public Sub New(ByVal column As Integer, ByVal ascending As Boolean)
        columnIndex = column
        ascendingOrder = ascending
    End Sub

    Public Function Compare(ByVal x As Object, ByVal y As Object) As Integer Implements
System.Collections.IComparer.Compare
        Dim leftRow As ListViewItem = CType(x, ListViewItem)
        Dim rightRow As ListViewItem = CType(y, ListViewItem)

        Dim leftText As String = If(leftRow.SubItems.Count > columnIndex,
leftRow.SubItems(columnIndex).Text, "")
        Dim rightText As String = If(rightRow.SubItems.Count > columnIndex,
rightRow.SubItems(columnIndex).Text, "")

        Dim answer As Integer
        If columnIndex = 2 Then
            answer = BytesOf(leftText).CompareTo(BytesOf(rightText))
        Else
            answer = String.Compare(leftText, rightText, StringComparison.OrdinalIgnoreCase)
        End If
```

```
        If Not ascendingOrder Then answer = -answer
        Return answer
    End Function

    Private Function BytesOf(ByVal sizeText As String) As Double
        Dim numberPart As String = ""
        For Each character As Char In sizeText
            If Char.IsDigit(character) OrElse character = "."c Then
                numberPart &= character
            Else
                Exit For
            End If
        Next

        Dim value As Double = 0
        Double.TryParse(numberPart, value)

        If sizeText.Contains("GB") Then Return value * 1073741824
        If sizeText.Contains("MB") Then Return value * 1048576
        If sizeText.Contains("KB") Then Return value * 1024
        Return value
    End Function
End Class

' ------------------------------------------------------
' preview
' ------------------------------------------------------
Private Sub lvFiles_SelectedIndexChanged(ByVal sender As Object, ByVal e As EventArgs) Handles
lvFiles.SelectedIndexChanged
    If lvFiles.SelectedItems.Count = 0 Then
        ShowPreview("")
        Return
    End If

    ShowPreview(CStr(lvFiles.SelectedItems(0).Tag))
End Sub

Private Sub lvFiles_DoubleClick(ByVal sender As Object, ByVal e As EventArgs) Handles
lvFiles.DoubleClick
    If lvFiles.SelectedItems.Count = 0 Then Return

    Dim chosenPath As String = CStr(lvFiles.SelectedItems(0).Tag)

    If Directory.Exists(chosenPath) Then
        NavigateTo(chosenPath, True)
        Return
    End If

    Try
        Process.Start(chosenPath)
    Catch ex As Exception
        lblStatus.Text = "This file cannot be opened : " & ex.Message
    End Try
End Sub

Private Sub ShowPreview(ByVal chosenPath As String)
    ReleasePicture()

    If chosenPath = "" Then
        lblPreviewTitle.Text = "PREVIEW"
        txtPreview.Visible = True
        txtPreview.Text = "Select a file or a folder to preview it here."
        Return
    End If

    Dim itemName As String = Path.GetFileName(chosenPath)
    If itemName = "" Then itemName = chosenPath
    lblPreviewTitle.Text = "PREVIEW  -  " & itemName

    If Directory.Exists(chosenPath) Then
        ShowFolderPreview(chosenPath)
        Return
    End If

    If Not File.Exists(chosenPath) Then
        txtPreview.Visible = True
        txtPreview.Text = "The selected item no longer exists."
        Return
    End If

    Dim info As New FileInfo(chosenPath)
    Dim extension As String = info.Extension.ToLowerInvariant()

    If IsOneOf(extension, imageTypes) Then
        If ShowImagePreview(chosenPath) Then Return
    End If
```

```
    txtPreview.Visible = True

    If IsOneOf(extension, textTypes) Then
        txtPreview.Text = ReadTextPreview(chosenPath, info)
    Else
        txtPreview.Text = "Name          : " & info.Name & vbCrLf & _
                          "Type          : " & TypeLabel(info) & vbCrLf & _
                          "Size          : " & SizeLabel(info.Length) & vbCrLf & _
                          "Created       : " & info.CreationTime.ToString("MMM d, yyyy  h:mm tt") &
vbCrLf & _
                          "Last modified : " & info.LastWriteTime.ToString("MMM d, yyyy  h:mm tt") &
vbCrLf & _
                          "Attributes    : " & info.Attributes.ToString() & vbCrLf & vbCrLf & _
                          "There is no built-in preview for this file type."
    End If
End Sub

Private Function IsOneOf(ByVal value As String, ByVal choices As String()) As Boolean
    For Each choice As String In choices
        If value = choice Then Return True
    Next
    Return False
End Function

Private Function ShowImagePreview(ByVal filePath As String) As Boolean
    Try
        Using stream As New FileStream(filePath, FileMode.Open, FileAccess.Read)
            Using source As Image = Image.FromStream(stream)
                pbPreview.Image = New Bitmap(source)
            End Using
        End Using

        txtPreview.Visible = False
        pbPreview.Visible = True
        lblStatus.Text = "Image preview : " & Path.GetFileName(filePath)
        Return True
    Catch ex As Exception
        txtPreview.Visible = True
        pbPreview.Visible = False
        txtPreview.Text = "The image cannot be previewed." & vbCrLf & ex.Message
        Return True
    End Try
End Function

Private Sub ShowFolderPreview(ByVal folderPath As String)
    Dim summary As New StringBuilder()
    Dim folderCount As Integer = 0
    Dim fileCount As Integer = 0
    Dim totalSize As Long = 0

    Try
        For Each childFolder As String In Directory.GetDirectories(folderPath)
            folderCount += 1
        Next

        For Each childFile As String In Directory.GetFiles(folderPath)
            Dim childInfo As New FileInfo(childFile)
            fileCount += 1
            totalSize += childInfo.Length
        Next
    Catch ex As Exception
        summary.AppendLine("Some items cannot be counted : " & ex.Message)
    End Try

    Dim parentPath As String = Path.GetDirectoryName(folderPath)
    If parentPath Is Nothing Then parentPath = folderPath

    summary.AppendLine("Folder        : " & folderPath)
    summary.AppendLine("Parent        : " & parentPath)
    summary.AppendLine("Sub folders   : " & folderCount.ToString())
    summary.AppendLine("Files         : " & fileCount.ToString())
    summary.AppendLine("Total size    : " & SizeLabel(totalSize))
    summary.AppendLine()
    summary.AppendLine("CONTENTS (first 40 items)")
    summary.AppendLine("------------------------------------------")

    Try
        Dim shown As Integer = 0
        For Each childFolder As String In Directory.GetDirectories(folderPath)
            summary.AppendLine("[folder]  " & Path.GetFileName(childFolder))
            shown += 1
            If shown >= 40 Then Exit For
        Next

        If shown < 40 Then
            For Each childFile As String In Directory.GetFiles(folderPath)
                Dim childInfo As New FileInfo(childFile)
```

```
                summary.AppendLine("          " & childInfo.Name & "   (" & SizeLabel(childInfo.Length)
& ")")
                shown += 1
                If shown >= 40 Then Exit For
            Next
        End If
    Catch ex As Exception
        summary.AppendLine(ex.Message)
    End Try

    txtPreview.Visible = True
    txtPreview.Text = summary.ToString()
End Sub

Private Function ReadTextPreview(ByVal filePath As String, ByVal info As FileInfo) As String
    Const characterLimit As Integer = 6000

    Try
        Dim buffer(characterLimit) As Char
        Dim text As New StringBuilder()

        Using reader As New StreamReader(filePath, Encoding.Default, True)
            Dim readNow As Integer = reader.Read(buffer, 0, characterLimit)
            While readNow > 0
                text.Append(New String(buffer, 0, readNow))
                readNow = reader.Read(buffer, 0, characterLimit)
            End While
        End Using

        Dim header As String = "Name : " & info.Name & "    Size : " & SizeLabel(info.Length) & vbCrLf &
 _
                               "Modified : " & info.LastWriteTime.ToString("MMM d, yyyy  h:mm tt") &
vbCrLf & _
                               "-----------------------------------------" & vbCrLf

        Dim body As String = text.ToString()
        If body.Length >= characterLimit Then
            body &= vbCrLf & vbCrLf & "...  only the first " & characterLimit.ToString() & " characters
are shown."
        End If

        Return header & body
    Catch ex As Exception
        Return "The text of this file cannot be shown." & vbCrLf & ex.Message
    End Try
End Function

Private Sub ClearPreview(ByVal message As String)
    ReleasePicture()
    pbPreview.Visible = False
    txtPreview.Visible = True
    txtPreview.Text = message
End Sub

Private Sub ReleasePicture()
    If pbPreview.Image IsNot Nothing Then
        Dim oldPicture As Image = pbPreview.Image
        pbPreview.Image = Nothing
        oldPicture.Dispose()
    End If
    pbPreview.Visible = False
End Sub

' ------------------------------------------------------------
' split container controls
' ------------------------------------------------------------
Private Sub ApplyDefaultLayout()
    splRoot.Panel1Collapsed = False
    splRoot.Panel2Collapsed = False
    splWorkspace.Panel1Collapsed = False
    splWorkspace.Panel2Collapsed = False
    splWorkspace.Orientation = Orientation.Horizontal

    btnTreeToggle.Text = "Hide folder tree"
    btnPreviewToggle.Text = "Hide preview pane"
    btnSwapLayout.Text = "Preview beside list"

    SetRatioDistance(splRoot, 0.20)
    SetRatioDistance(splWorkspace, 0.62)
    ShowSplitInfo()
End Sub

Private Sub SetRatioDistance(ByVal splitter As SplitContainer, ByVal ratio As Double)
    Dim available As Integer
    If splitter.Orientation = Orientation.Vertical Then
        available = splitter.ClientSize.Width
    Else
```

```
            available = splitter.ClientSize.Height
        End If

        Dim lowest As Integer = splitter.Panel1MinSize
        Dim highest As Integer = available - splitter.Panel2MinSize - splitter.SplitterWidth
        If highest <= lowest Then Return

        Dim requested As Integer = CInt(available * ratio)
        If requested < lowest Then requested = lowest
        If requested > highest Then requested = highest
        splitter.SplitterDistance = requested
    End Sub

    Private Sub ShowSplitInfo()
        lblSplitInfo.Text = "splitRoot : " & splRoot.SplitterDistance.ToString() & " px   |   " & _
                            "splitWorkspace : " & splWorkspace.SplitterDistance.ToString() & " px"
    End Sub

    Private Sub btnTreeToggle_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnTreeToggle.Click
        splRoot.Panel1Collapsed = Not splRoot.Panel1Collapsed
        btnTreeToggle.Text = If(splRoot.Panel1Collapsed, "Show folder tree", "Hide folder tree")
        lblStatus.Text = "splRoot.Panel1Collapsed = " & splRoot.Panel1Collapsed.ToString()
        ShowSplitInfo()
    End Sub

    Private Sub btnPreviewToggle_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnPreviewToggle.Click
        splWorkspace.Panel2Collapsed = Not splWorkspace.Panel2Collapsed
        btnPreviewToggle.Text = If(splWorkspace.Panel2Collapsed, "Show preview pane", "Hide preview pane")
        lblStatus.Text = "splWorkspace.Panel2Collapsed = " & splWorkspace.Panel2Collapsed.ToString()
        ShowSplitInfo()
    End Sub

    Private Sub btnSwapLayout_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnSwapLayout.Click
        Try
            If splWorkspace.Orientation = Orientation.Horizontal Then
                splWorkspace.Orientation = Orientation.Vertical
                btnSwapLayout.Text = "Preview below list"
                lblStatus.Text = "The preview is now beside the file list."
            Else
                splWorkspace.Orientation = Orientation.Horizontal
                btnSwapLayout.Text = "Preview beside list"
                lblStatus.Text = "The preview is now below the file list."
            End If

            SetRatioDistance(splWorkspace, 0.62)
            ShowSplitInfo()
        Catch ex As Exception
            lblStatus.Text = "The layout cannot be changed : " & ex.Message
        End Try
    End Sub

    Private Sub btnLockSplitters_Click(ByVal sender As Object, ByVal e As EventArgs) Handles
    btnLockSplitters.Click
        splittersLocked = Not splittersLocked
        splRoot.IsSplitterFixed = splittersLocked
        splWorkspace.IsSplitterFixed = splittersLocked
        btnLockSplitters.Text = If(splittersLocked, "Unlock splitters", "Lock splitters")
        lblStatus.Text = "IsSplitterFixed = " & splittersLocked.ToString()
    End Sub

    Private Sub splRoot_SplitterMoved(ByVal sender As Object, ByVal e As SplitterEventArgs) Handles
    splRoot.SplitterMoved
        ShowSplitInfo()
        lblStatus.Text = "Folder tree width set to " & splRoot.SplitterDistance.ToString() & " pixels."
    End Sub

    Private Sub splWorkspace_SplitterMoved(ByVal sender As Object, ByVal e As SplitterEventArgs) Handles
    splWorkspace.SplitterMoved
        ShowSplitInfo()
        lblStatus.Text = "Preview size set to " & splWorkspace.SplitterDistance.ToString() & " pixels."
    End Sub
End Class
```

Notes

The preview pane keeps the memory safe: every new picture disposes the previous bitmap, and only the first 6000 characters of a text file are read.

Activity 13

Activity 13 - Integrative Project - School Records with MDI Forms

Project : IntegProgProject2026 | Author : Mangalili, Fhetz | BSIT 3B

Description

The final activity is the integrative project. A main MDI form holds a menu, a tool strip that shows the name of the user and a live date and time, and a status bar. Four child forms are opened inside the main form: Schedules, Quotes, Bible Verses and About. The record forms read and write their data through a DataTable that is saved to an XML file, and the About form shows the picture of the programmer.

Objectives

- Build an MDI parent form with child forms.
- Store and load records with DataTable, ReadXml and WriteXml.
- Use a MenuStrip, a ToolStrip with a Timer and a StatusStrip.

Source Code

frmMain.vb

```
 Public Class frmMain

    Private Sub LogLine(ByVal text As String)
        Try
            IO.File.AppendAllText(IO.Path.Combine(Application.StartupPath, "forms_log.txt"), _
                Date.Now.ToString("HH:mm:ss") & "  " & text & vbCrLf)
        Catch
        End Try
    End Sub

    Private Sub frmMain_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
        Try
            toolstripUserInfo.Text = "UserInfo: Mangalili, Fhetz  |  BSIT 3B"
            UpdateDateTime()
            Timer1.Interval = 1000
            Timer1.Start()
            lblStatusBar.Text = "Choose a form from the menu."
            LogLine("main form loaded")
        Catch ex As Exception
            LogLine("main load failed: " & ex.Message)
        End Try
    End Sub

    Private Sub Timer1_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles Timer1.Tick
        Try
            UpdateDateTime()
        Catch
        End Try
    End Sub

    Private Sub UpdateDateTime()
        toolstripCurrentDate.Text = "CurrentDate: " & Date.Now.ToString("dddd, d MMMM yyyy")
        toolstripCurrentTime.Text = "Current Time: " & Date.Now.ToString("h:mm:ss tt")
    End Sub

    Private Sub SchedulesToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles SchedulesToolStripMenuItem.Click
        ShowChild(GetType(frmSchedules))
    End Sub

    Private Sub AboutToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles AboutToolStripMenuItem.Click
        ShowChild(GetType(frmAbout))
    End Sub

    Private Sub QuotesToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles QuotesToolStripMenuItem.Click
        ShowChild(GetType(frmQuotes))
    End Sub

    Private Sub BibleVerseToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e As
    System.EventArgs) Handles BibleVerseToolStripMenuItem.Click
        ShowChild(GetType(frmBibleVerses))
    End Sub
```

```
    Private Sub ExitToolStripMenuItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles ExitToolStripMenuItem.Click
        If MessageBox.Show("Close the program?", "Exit", MessageBoxButtons.YesNo, MessageBoxIcon.Question) =
     DialogResult.Yes Then
            Me.Close()
        End If
    End Sub

    Private Sub ShowChild(ByVal formType As Type)
        LogLine("opening " & formType.Name)
        Try
            For Each openForm As Form In Me.MdiChildren
                If openForm.GetType() Is formType Then
                    openForm.Activate()
                    lblStatusBar.Text = "Form: " & openForm.Text
                    LogLine(formType.Name & " already open - activated")
                    Return
                End If
            Next
            Dim child As Form = CType(Activator.CreateInstance(formType), Form)
            child.MdiParent = Me
            child.Show()
            child.WindowState = FormWindowState.Maximized
            lblStatusBar.Text = "Form: " & child.Text
            LogLine(formType.Name & " shown inside main form, Visible=" & child.Visible.ToString())
        Catch ex As Exception
            LogLine(formType.Name & " FAILED: " & ex.GetType().Name & " - " & ex.Message)
            Try
                MessageBox.Show("The form could not be opened." & vbCrLf & vbCrLf & _
                    "Form: " & formType.Name & vbCrLf & vbCrLf & _
                    "Details: " & ex.Message, "Form Error", MessageBoxButtons.OK, MessageBoxIcon.Error)
            Catch
            End Try
        End Try
    End Sub

End Class
```

frmSchedules.vb

```
Imports System.Data.OleDb

Public Class frmSchedules

    Private selectedId As Integer = 0
    Private isLoading As Boolean = False
    Private store As DataTable
    Private nextId As Integer = 1

    Private Sub LogStep(ByVal text As String)
        Try
            IO.File.AppendAllText(IO.Path.Combine(Application.StartupPath, "forms_log.txt"), _
                Date.Now.ToString("HH:mm:ss") & "  [frmSchedules] " & text & vbCrLf)
        Catch
        End Try
    End Sub

    Private Function XmlPath() As String
        Return IO.Path.Combine(Application.StartupPath, "projectDB.xml")
    End Function

    Private Function SdfPath() As String
        Return IO.Path.Combine(Application.StartupPath, "projectDB.sdf")
    End Function

    Private Function HasSqlCe() As Boolean
        Try
            Using key As Microsoft.Win32.RegistryKey =
    Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOFTWARE\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
                If key IsNot Nothing Then Return True
            End Using
            Using key As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOF
    TWARE\Wow6432Node\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
                If key IsNot Nothing Then Return True
            End Using
        Catch
        End Try
        Return False
    End Function

    Private Sub frmSchedules_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    MyBase.Load
        LogStep("Load")
        Try
            SetupStore()
        Catch ex As Exception
            LogStep("SetupStore failed: " & ex.Message)
        End Try
```

```
    Try
        ClearFields()
    Catch
    End Try
    RefreshGrid()
End Sub

Private Sub SetupStore()
    store = New DataTable("Schedules")
    store.Columns.Add("ScheduleID", GetType(Integer))
    store.Columns.Add("CourseID", GetType(String))
    store.Columns.Add("TeacherID", GetType(String))
    store.Columns.Add("Day", GetType(String))
    store.Columns.Add("Time", GetType(String))
    store.Columns.Add("Room", GetType(String))
    If IO.File.Exists(XmlPath()) Then
        Try
            store.ReadXml(XmlPath())
            LogStep("loaded rows from xml: " & store.Rows.Count.ToString())
        Catch ex As Exception
            LogStep("xml unreadable, reseeding: " & ex.Message)
            store.Clear()
        End Try
    End If
    If store.Rows.Count = 0 Then
        SeedSampleData()
        SaveStore()
        LogStep("seeded sample data")
    End If
    nextId = 1
    For Each row As DataRow In store.Rows
        Dim id As Integer = Convert.ToInt32(row.Item("ScheduleID"))
        If id >= nextId Then nextId = id + 1
    Next
    lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & " record(s)"
End Sub

Private Sub SaveStore()
    Try
        store.AcceptChanges()
        store.WriteXml(XmlPath())
    Catch ex As Exception
        LogStep("WriteXml failed: " & ex.Message)
    End Try
    SyncToSdf()
End Sub

Private Sub SyncToSdf()
    If Not HasSqlCe() Then Return
    Try
        If Not IO.File.Exists(SdfPath()) Then
            Dim catalog As Object = CreateObject("ADOX.Catalog")
            catalog.Create("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" & SdfPath())
            catalog = Nothing
        End If
        Using conn As New OleDbConnection("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" &
SdfPath() & ";Persist Security Info=False;")
            conn.Open()
            Using cmd As New OleDbCommand("SELECT COUNT(*) FROM [Schedules]", conn)
                cmd.ExecuteScalar()
            End Using
            Using cmd As New OleDbCommand("DELETE FROM [Schedules]", conn)
                cmd.ExecuteNonQuery()
            End Using
            For Each row As DataRow In store.Rows
                Using cmd As New OleDbCommand("INSERT INTO [Schedules] ([CourseID], [TeacherID], [Day],
[Time], [Room]) VALUES (?, ?, ?, ?, ?)", conn)
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("CourseID")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("TeacherID")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Day")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Time")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Room")))
                    cmd.ExecuteNonQuery()
                End Using
            Next
        End Using
    Catch ex As Exception
        LogStep("sdf mirror skipped: " & ex.Message)
    End Try
End Sub

Private Sub SeedSampleData()
    Dim rows As String() = New String() { _
        "IT301|T-005|Monday|07:30|Room 205", _
        "IT302|T-006|Monday|11:00|Lab 3", _
        "CS201|T-007|Tuesday|09:30|Room 206", _
        "IT303|T-005|Wednesday|08:00|Lab 4", _
```

```
        "CS202|T-006|Thursday|13:30|Room 207", _
        "IT304|T-007|Friday|10:00|Room 208", _
        "CS203|T-005|Saturday|09:00|Lab 5"
        }
    For Each line As String In rows
        Dim parts As String() = line.Split("|"c)
        If parts.Length = 5 Then
            Dim row As DataRow = store.NewRow()
            row.Item("ScheduleID") = nextId
            row.Item("CourseID") = parts(0)
            row.Item("TeacherID") = parts(1)
            row.Item("Day") = parts(2)
            row.Item("Time") = parts(3)
            row.Item("Room") = parts(4)
            store.Rows.Add(row)
            nextId += 1
        End If
    Next
End Sub

Private Sub RefreshGrid()
    Try
        If store Is Nothing Then Return
        Dim view As New DataTable("Schedules")
        view = store.Copy()
        dataGridView1.DataSource = view
        FormatGrid()
        lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & "
record(s)"
    Catch ex As Exception
        LogStep("RefreshGrid failed: " & ex.Message)
    End Try
End Sub

Private Sub FormatGrid()
    If dataGridView1.Columns.Count = 0 Then Return
    With dataGridView1
        .Columns("ScheduleID").HeaderText = "ID"
        .Columns("CourseID").HeaderText = "Course ID"
        .Columns("TeacherID").HeaderText = "Teacher ID"
        .Columns("Day").HeaderText = "Day"
        .Columns("Time").HeaderText = "Time"
        .Columns("Room").HeaderText = "Room"
    End With
End Sub

Private Sub addButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
addButton.Click
    Try
        If Not IsValidInput() Then Return
        Dim row As DataRow = store.NewRow()
        row.Item("ScheduleID") = nextId
        row.Item("CourseID") = txtCourseID.Text.Trim()
        row.Item("TeacherID") = txtTeacherID.Text.Trim()
        row.Item("Day") = cboDay.Text.Trim()
        row.Item("Time") = dtpTime.Value.ToString("HH:mm")
        row.Item("Room") = txtRoom.Text.Trim()
        store.Rows.Add(row)
        nextId += 1
        SaveStore()
        MessageBox.Show("The record was saved." & vbCrLf & vbCrLf & "New ID: " & (nextId -
1).ToString(), _
            "Record Added", MessageBoxButtons.OK, MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Add failed: " & ex.Message)
        MessageBox.Show("The record could not be saved." & vbCrLf & vbCrLf & ex.Message, _
            "Save Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub updateButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
updateButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If Not IsValidInput() Then Return
        Dim found As DataRow = Nothing
        For Each row As DataRow In store.Rows
            If Convert.ToInt32(row.Item("ScheduleID")) = selectedId Then
                found = row
                Exit For
            End If
```

```
        Next
        If found Is Nothing Then
            MessageBox.Show("That record no longer exists.", "Record Not Found", _
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            RefreshGrid()
            Return
        End If
        found.Item("CourseID") = txtCourseID.Text.Trim()
        found.Item("TeacherID") = txtTeacherID.Text.Trim()
        found.Item("Day") = cboDay.Text.Trim()
        found.Item("Time") = dtpTime.Value.ToString("HH:mm")
        found.Item("Room") = txtRoom.Text.Trim()
        SaveStore()
        MessageBox.Show("The record was updated.", "Record Updated", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Update failed: " & ex.Message)
        MessageBox.Show("The record could not be updated." & vbCrLf & vbCrLf & ex.Message, _
            "Update Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub deleteButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
deleteButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If MessageBox.Show("Delete this record?" & vbCrLf & vbCrLf & "ID: " & selectedId.ToString(), _
                "Confirm Delete", MessageBoxButtons.YesNo, MessageBoxIcon.Question) <> DialogResult.Yes
Then
            Return
        End If
        For i As Integer = store.Rows.Count - 1 To 0 Step -1
            If Convert.ToInt32(store.Rows(i).Item("ScheduleID")) = selectedId Then
                store.Rows.RemoveAt(i)
            End If
        Next
        SaveStore()
        MessageBox.Show("The record was deleted.", "Record Deleted", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Delete failed: " & ex.Message)
        MessageBox.Show("The record could not be deleted." & vbCrLf & vbCrLf & ex.Message, _
            "Delete Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub clearButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
clearButton.Click
    ClearFields()
End Sub

Private Sub exitButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
exitButton.Click
    Me.Close()
End Sub

Private Sub dataGridView1_SelectionChanged(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles dataGridView1.SelectionChanged
    If isLoading Then Return
    If dataGridView1.CurrentRow Is Nothing Then Return
    FillFromRow()
End Sub

Private Sub FillFromRow()
    Dim row As DataGridViewRow = dataGridView1.CurrentRow
    If row Is Nothing Then Return
    If row.Cells("ScheduleID").Value Is Nothing Then Return
    If row.Cells("ScheduleID").Value Is DBNull.Value Then Return
    isLoading = True
    Try
        selectedId = Convert.ToInt32(row.Cells("ScheduleID").Value)
        txtScheduleID.Text = selectedId.ToString()
        txtCourseID.Text = Convert.ToString(row.Cells("CourseID").Value)
        txtTeacherID.Text = Convert.ToString(row.Cells("TeacherID").Value)
        cboDay.Text = Convert.ToString(row.Cells("Day").Value)
        Dim TimeText As String = Convert.ToString(row.Cells("Time").Value)
        Dim TimeParsed As Date
        If Date.TryParse(DateTime.Today.ToShortDateString() & " " & TimeText, TimeParsed) Then
            dtpTime.Value = TimeParsed
```

```
            End If
            txtRoom.Text = Convert.ToString(row.Cells("Room").Value)
        Catch ex As Exception
            LogStep("FillFromRow failed: " & ex.Message)
        Finally
            isLoading = False
        End Try
    End Sub

    Private Sub ClearFields()
        isLoading = True
        Try
            dataGridView1.CurrentCell = Nothing
        Catch
        End Try
        selectedId = 0
        txtScheduleID.Clear()
        txtCourseID.Clear()
        txtTeacherID.Clear()
        cboDay.SelectedIndex = -1
        dtpTime.Value = DateTime.Today.AddHours(8)
        txtRoom.Clear()
        txtCourseID.Focus()
        isLoading = False
    End Sub

    Private Function IsValidInput() As Boolean
        If txtCourseID.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Course ID.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtCourseID.Focus()
            Return False
        End If
        If txtTeacherID.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Teacher ID.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtTeacherID.Focus()
            Return False
        End If
        If cboDay.SelectedIndex < 0 Then
            MessageBox.Show("Please choose the Day.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            cboDay.Focus()
            Return False
        End If
        If txtRoom.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Room.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtRoom.Focus()
            Return False
        End If
        Return True
    End Function


    Private Function IntValue(ByVal text As String) As Integer
        Dim value As Integer = 0
        If Integer.TryParse(text.Trim(), value) Then Return value
        Return 0
    End Function

    Private Function DateValue(ByVal text As String) As Date
        Dim parts As String() = text.Split("-"c)
        If parts.Length = 3 Then
            Dim y As Integer = 0
            Dim m As Integer = 0
            Dim d As Integer = 0
            If Integer.TryParse(parts(0), y) AndAlso Integer.TryParse(parts(1), m) AndAlso
    Integer.TryParse(parts(2), d) Then
                If y >= 1 AndAlso m >= 1 AndAlso m <= 12 AndAlso d >= 1 AndAlso d <= 31 Then
                    Return New Date(y, m, d)
                End If
            End If
        End If
        Return Date.Today
    End Function

End Class
```

frmQuotes.vb

```
Imports System.Data.OleDb

Public Class frmQuotes

    Private selectedId As Integer = 0
    Private isLoading As Boolean = False
    Private store As DataTable
```

```
Private nextId As Integer = 1

Private Sub LogStep(ByVal text As String)
    Try
        IO.File.AppendAllText(IO.Path.Combine(Application.StartupPath, "forms_log.txt"), _
            Date.Now.ToString("HH:mm:ss") & "  [frmQuotes] " & text & vbCrLf)
    Catch
    End Try
End Sub

Private Function XmlPath() As String
    Return IO.Path.Combine(Application.StartupPath, "projectDB.xml")
End Function

Private Function SdfPath() As String
    Return IO.Path.Combine(Application.StartupPath, "projectDB.sdf")
End Function

Private Function HasSqlCe() As Boolean
    Try
        Using key As Microsoft.Win32.RegistryKey =
Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOFTWARE\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
            If key IsNot Nothing Then Return True
        End Using
        Using key As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOF
TWARE\Wow6432Node\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
            If key IsNot Nothing Then Return True
        End Using
    Catch
    End Try
    Return False
End Function

Private Sub frmQuotes_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
MyBase.Load
    LogStep("Load")
    Try
        SetupStore()
    Catch ex As Exception
        LogStep("SetupStore failed: " & ex.Message)
    End Try
    Try
        ClearFields()
    Catch
    End Try
    RefreshGrid()
End Sub

Private Sub SetupStore()
    store = New DataTable("Quotes")
    store.Columns.Add("QuoteID", GetType(Integer))
    store.Columns.Add("Content", GetType(String))
    store.Columns.Add("Author", GetType(String))
    store.Columns.Add("Source", GetType(String))
    store.Columns.Add("QuoteDate", GetType(Date))
    If IO.File.Exists(XmlPath()) Then
        Try
            store.ReadXml(XmlPath())
            LogStep("loaded rows from xml: " & store.Rows.Count.ToString())
        Catch ex As Exception
            LogStep("xml unreadable, reseeding: " & ex.Message)
            store.Clear()
        End Try
    End If
    If store.Rows.Count = 0 Then
        SeedSampleData()
        SaveStore()
        LogStep("seeded sample data")
    End If
    nextId = 1
    For Each row As DataRow In store.Rows
        Dim id As Integer = Convert.ToInt32(row.Item("QuoteID"))
        If id >= nextId Then nextId = id + 1
    Next
    lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & " record(s)"
End Sub

Private Sub SaveStore()
    Try
        store.AcceptChanges()
        store.WriteXml(XmlPath())
    Catch ex As Exception
        LogStep("WriteXml failed: " & ex.Message)
    End Try
    SyncToSdf()
End Sub
```

```
Private Sub SyncToSdf()
    If Not HasSqlCe() Then Return
    Try
        If Not IO.File.Exists(SdfPath()) Then
            Dim catalog As Object = CreateObject("ADOX.Catalog")
            catalog.Create("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" & SdfPath())
            catalog = Nothing
        End If
        Using conn As New OleDbConnection("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" &
SdfPath() & ";Persist Security Info=False;")
            conn.Open()
            Using cmd As New OleDbCommand("SELECT COUNT(*) FROM [Quotes]", conn)
                cmd.ExecuteScalar()
            End Using
            Using cmd As New OleDbCommand("DELETE FROM [Quotes]", conn)
                cmd.ExecuteNonQuery()
            End Using
            For Each row As DataRow In store.Rows
                Using cmd As New OleDbCommand("INSERT INTO [Quotes] ([Content], [Author], [Source],
[QuoteDate]) VALUES (?, ?, ?, ?)", conn)
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Content")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Author")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Source")))
                    cmd.Parameters.AddWithValue("?", Convert.ToDateTime(row.Item("QuoteDate")))
                    cmd.ExecuteNonQuery()
                End Using
            Next
        End Using
    Catch ex As Exception
        LogStep("sdf mirror skipped: " & ex.Message)
    End Try
End Sub

Private Sub SeedSampleData()
    Dim rows As String() = New String() { _
        "Simplicity is the soul of efficiency.|Austin Freeman|The Social Welfare of the
Blind|1912-01-01", _
        "First, solve the problem. Then, write the code.|John Johnson|Tech talk|1980-05-20", _
        "Experience is the name everyone gives to their mistakes.|Oscar Wilde|Lady Windermere
Fan|1892-02-20", _
        "Code is like humor. When you have to explain it, it is bad.|Cory House|Blog post|2015-03-11", _
        "Make it work, make it right, make it fast.|Kent Beck|Extreme Programming|1999-10-05", _
        "The best error message is the one that never shows up.|Thomas Fuchs|Conference talk|2011-07-19"
        }
    For Each line As String In rows
        Dim parts As String() = line.Split("|"c)
        If parts.Length = 4 Then
            Dim row As DataRow = store.NewRow()
            row.Item("QuoteID") = nextId
            row.Item("Content") = parts(0)
            row.Item("Author") = parts(1)
            row.Item("Source") = parts(2)
            row.Item("QuoteDate") = DateValue(parts(3))
            store.Rows.Add(row)
            nextId += 1
        End If
    Next
End Sub

Private Sub RefreshGrid()
    Try
        If store Is Nothing Then Return
        Dim view As New DataTable("Quotes")
        view = store.Copy()
        dataGridView1.DataSource = view
        FormatGrid()
        lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & "
record(s)"
    Catch ex As Exception
        LogStep("RefreshGrid failed: " & ex.Message)
    End Try
End Sub

Private Sub FormatGrid()
    If dataGridView1.Columns.Count = 0 Then Return
    With dataGridView1
        .Columns("QuoteID").HeaderText = "ID"
        .Columns("Content").HeaderText = "Content"
        .Columns("Author").HeaderText = "Author"
        .Columns("Source").HeaderText = "Source"
        .Columns("QuoteDate").HeaderText = "Quote Date"
    End With
End Sub

Private Sub addButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
addButton.Click
    Try
```

```
        If Not IsValidInput() Then Return
        Dim row As DataRow = store.NewRow()
        row.Item("QuoteID") = nextId
        row.Item("Content") = txtContent.Text.Trim()
        row.Item("Author") = txtAuthor.Text.Trim()
        row.Item("Source") = txtSource.Text.Trim()
        row.Item("QuoteDate") = dtpQuoteDate.Value.Date
        store.Rows.Add(row)
        nextId += 1
        SaveStore()
        MessageBox.Show("The record was saved." & vbCrLf & vbCrLf & "New ID: " & (nextId -
1).ToString(), _
            "Record Added", MessageBoxButtons.OK, MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Add failed: " & ex.Message)
        MessageBox.Show("The record could not be saved." & vbCrLf & vbCrLf & ex.Message, _
            "Save Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub updateButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
updateButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If Not IsValidInput() Then Return
        Dim found As DataRow = Nothing
        For Each row As DataRow In store.Rows
            If Convert.ToInt32(row.Item("QuoteID")) = selectedId Then
                found = row
                Exit For
            End If
        Next
        If found Is Nothing Then
            MessageBox.Show("That record no longer exists.", "Record Not Found", _
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            RefreshGrid()
            Return
        End If
        found.Item("Content") = txtContent.Text.Trim()
        found.Item("Author") = txtAuthor.Text.Trim()
        found.Item("Source") = txtSource.Text.Trim()
        found.Item("QuoteDate") = dtpQuoteDate.Value.Date
        SaveStore()
        MessageBox.Show("The record was updated.", "Record Updated", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Update failed: " & ex.Message)
        MessageBox.Show("The record could not be updated." & vbCrLf & vbCrLf & ex.Message, _
            "Update Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub deleteButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
deleteButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If MessageBox.Show("Delete this record?" & vbCrLf & vbCrLf & "ID: " & selectedId.ToString(), _
                "Confirm Delete", MessageBoxButtons.YesNo, MessageBoxIcon.Question) <> DialogResult.Yes
Then
            Return
        End If
        For i As Integer = store.Rows.Count - 1 To 0 Step -1
            If Convert.ToInt32(store.Rows(i).Item("QuoteID")) = selectedId Then
                store.Rows.RemoveAt(i)
            End If
        Next
        SaveStore()
        MessageBox.Show("The record was deleted.", "Record Deleted", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Delete failed: " & ex.Message)
        MessageBox.Show("The record could not be deleted." & vbCrLf & vbCrLf & ex.Message, _
            "Delete Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
```

```
    End Try
End Sub

Private Sub clearButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
clearButton.Click
    ClearFields()
End Sub

Private Sub exitButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
exitButton.Click
    Me.Close()
End Sub

Private Sub dataGridView1_SelectionChanged(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles dataGridView1.SelectionChanged
    If isLoading Then Return
    If dataGridView1.CurrentRow Is Nothing Then Return
    FillFromRow()
End Sub

Private Sub FillFromRow()
    Dim row As DataGridViewRow = dataGridView1.CurrentRow
    If row Is Nothing Then Return
    If row.Cells("QuoteID").Value Is Nothing Then Return
    If row.Cells("QuoteID").Value Is DBNull.Value Then Return
    isLoading = True
    Try
        selectedId = Convert.ToInt32(row.Cells("QuoteID").Value)
        txtQuoteID.Text = selectedId.ToString()
        txtContent.Text = Convert.ToString(row.Cells("Content").Value)
        txtAuthor.Text = Convert.ToString(row.Cells("Author").Value)
        txtSource.Text = Convert.ToString(row.Cells("Source").Value)
        Dim QuoteDateDate As Date
        If Date.TryParse(Convert.ToString(row.Cells("QuoteDate").Value), QuoteDateDate) Then
            dtpQuoteDate.Value = QuoteDateDate
        End If
    Catch ex As Exception
        LogStep("FillFromRow failed: " & ex.Message)
    Finally
        isLoading = False
    End Try
End Sub

Private Sub ClearFields()
    isLoading = True
    Try
        dataGridView1.CurrentCell = Nothing
    Catch
    End Try
    selectedId = 0
    txtQuoteID.Clear()
    txtContent.Clear()
    txtAuthor.Clear()
    txtSource.Clear()
    dtpQuoteDate.Value = Date.Today
    txtContent.Focus()
    isLoading = False
End Sub

Private Function IsValidInput() As Boolean
    If txtContent.Text.Trim() = "" Then
        MessageBox.Show("Please enter the Content.", "Missing Information", MessageBoxButtons.OK,
MessageBoxIcon.Exclamation)
        txtContent.Focus()
        Return False
    End If
    If txtAuthor.Text.Trim() = "" Then
        MessageBox.Show("Please enter the Author.", "Missing Information", MessageBoxButtons.OK,
MessageBoxIcon.Exclamation)
        txtAuthor.Focus()
        Return False
    End If
    Return True
End Function


Private Function IntValue(ByVal text As String) As Integer
    Dim value As Integer = 0
    If Integer.TryParse(text.Trim(), value) Then Return value
    Return 0
End Function

Private Function DateValue(ByVal text As String) As Date
    Dim parts As String() = text.Split("-"c)
    If parts.Length = 3 Then
        Dim y As Integer = 0
        Dim m As Integer = 0
```

```
            Dim d As Integer = 0
            If Integer.TryParse(parts(0), y) AndAlso Integer.TryParse(parts(1), m) AndAlso
        Integer.TryParse(parts(2), d) Then
                If y >= 1 AndAlso m >= 1 AndAlso m <= 12 AndAlso d >= 1 AndAlso d <= 31 Then
                    Return New Date(y, m, d)
                End If
            End If
        End If
        Return Date.Today
    End Function

End Class
```

frmBibleVerses.vb

```
Imports System.Data.OleDb

Public Class frmBibleVerses

    Private selectedId As Integer = 0
    Private isLoading As Boolean = False
    Private store As DataTable
    Private nextId As Integer = 1

    Private Sub LogStep(ByVal text As String)
        Try
            IO.File.AppendAllText(IO.Path.Combine(Application.StartupPath, "forms_log.txt"), _
                Date.Now.ToString("HH:mm:ss") & "  [frmBibleVerses] " & text & vbCrLf)
        Catch
        End Try
    End Sub

    Private Function XmlPath() As String
        Return IO.Path.Combine(Application.StartupPath, "projectDB.xml")
    End Function

    Private Function SdfPath() As String
        Return IO.Path.Combine(Application.StartupPath, "projectDB.sdf")
    End Function

    Private Function HasSqlCe() As Boolean
        Try
            Using key As Microsoft.Win32.RegistryKey =
    Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOFTWARE\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
                If key IsNot Nothing Then Return True
            End Using
            Using key As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine.OpenSubKey("SOF
    TWARE\Wow6432Node\Classes\Microsoft.SQLSERVER.CE.OLEDB.3.5")
                If key IsNot Nothing Then Return True
            End Using
        Catch
        End Try
        Return False
    End Function

    Private Sub frmBibleVerses_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
    MyBase.Load
        LogStep("Load")
        Try
            SetupStore()
        Catch ex As Exception
            LogStep("SetupStore failed: " & ex.Message)
        End Try
        Try
            ClearFields()
        Catch
        End Try
        RefreshGrid()
    End Sub

    Private Sub SetupStore()
        store = New DataTable("BibleVerses")
        store.Columns.Add("VerseID", GetType(Integer))
        store.Columns.Add("Book", GetType(String))
        store.Columns.Add("Chapter", GetType(Integer))
        store.Columns.Add("Verse", GetType(Integer))
        store.Columns.Add("Content", GetType(String))
        store.Columns.Add("Translation", GetType(String))
        store.Columns.Add("Remarks", GetType(String))
        store.Columns.Add("Category", GetType(String))
        If IO.File.Exists(XmlPath()) Then
            Try
                store.ReadXml(XmlPath())
                LogStep("loaded rows from xml: " & store.Rows.Count.ToString())
            Catch ex As Exception
                LogStep("xml unreadable, reseeding: " & ex.Message)
                store.Clear()
            End Try
```

```
    End If
    If store.Rows.Count = 0 Then
        SeedSampleData()
        SaveStore()
        LogStep("seeded sample data")
    End If
    nextId = 1
    For Each row As DataRow In store.Rows
        Dim id As Integer = Convert.ToInt32(row.Item("VerseID"))
        If id >= nextId Then nextId = id + 1
    Next
    lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & " record(s)"
End Sub

Private Sub SaveStore()
    Try
        store.AcceptChanges()
        store.WriteXml(XmlPath())
    Catch ex As Exception
        LogStep("WriteXml failed: " & ex.Message)
    End Try
    SyncToSdf()
End Sub

Private Sub SyncToSdf()
    If Not HasSqlCe() Then Return
    Try
        If Not IO.File.Exists(SdfPath()) Then
            Dim catalog As Object = CreateObject("ADOX.Catalog")
            catalog.Create("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" & SdfPath())
            catalog = Nothing
        End If
        Using conn As New OleDbConnection("Provider=Microsoft.SQLSERVER.CE.OLEDB.3.5;Data Source=" &
SdfPath() & ";Persist Security Info=False;")
            conn.Open()
            Using cmd As New OleDbCommand("SELECT COUNT(*) FROM [BibleVerses]", conn)
                cmd.ExecuteScalar()
            End Using
            Using cmd As New OleDbCommand("DELETE FROM [BibleVerses]", conn)
                cmd.ExecuteNonQuery()
            End Using
            For Each row As DataRow In store.Rows
                Using cmd As New OleDbCommand("INSERT INTO [BibleVerses] ([Book], [Chapter], [Verse],
[Content], [Translation], [Remarks], [Category]) VALUES (?, ?, ?, ?, ?, ?, ?)", conn)
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Book")))
                    cmd.Parameters.AddWithValue("?", IntValue(Convert.ToString(row.Item("Chapter"))))
                    cmd.Parameters.AddWithValue("?", IntValue(Convert.ToString(row.Item("Verse"))))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Content")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Translation")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Remarks")))
                    cmd.Parameters.AddWithValue("?", Convert.ToString(row.Item("Category")))
                    cmd.ExecuteNonQuery()
                End Using
            Next
        End Using
    Catch ex As Exception
        LogStep("sdf mirror skipped: " & ex.Message)
    End Try
End Sub

Private Sub SeedSampleData()
    Dim rows As String() = New String() { _
        "Genesis|1|1|In the beginning God created the heavens and the earth.|NIV|Creation|Old
Testament", _
        "Exodus|20|3|You shall have no other gods before me.|NIV|Commandments|Old Testament", _
        "Joshua|1|9|Be strong and courageous. Do not be afraid.|NIV|Courage|History", _
        "Isaiah|41|10|So do not fear, for I am with you.|NIV|Comfort|Prophecy", _
        "Matthew|5|9|Blessed are the peacemakers.|NIV|Peace|Gospel", _
        "James|1|22|Do not merely listen to the word, and so deceive yourselves. Do what it
says.|NIV|Action|Epistle"
        }
    For Each line As String In rows
        Dim parts As String() = line.Split("|"c)
        If parts.Length = 7 Then
            Dim row As DataRow = store.NewRow()
            row.Item("VerseID") = nextId
            row.Item("Book") = parts(0)
            row.Item("Chapter") = IntValue(parts(1))
            row.Item("Verse") = IntValue(parts(2))
            row.Item("Content") = parts(3)
            row.Item("Translation") = parts(4)
            row.Item("Remarks") = parts(5)
            row.Item("Category") = parts(6)
            store.Rows.Add(row)
            nextId += 1
        End If
    Next
```

```
End Sub

Private Sub RefreshGrid()
    Try
        If store Is Nothing Then Return
        Dim view As New DataTable("BibleVerses")
        view = store.Copy()
        dataGridView1.DataSource = view
        FormatGrid()
        lblDatabase.Text = "Storage: " & XmlPath() & "   |   " & store.Rows.Count.ToString() & "
record(s)"
    Catch ex As Exception
        LogStep("RefreshGrid failed: " & ex.Message)
    End Try
End Sub

Private Sub FormatGrid()
    If dataGridView1.Columns.Count = 0 Then Return
    With dataGridView1
        .Columns("VerseID").HeaderText = "ID"
        .Columns("Book").HeaderText = "Book"
        .Columns("Chapter").HeaderText = "Chapter"
        .Columns("Verse").HeaderText = "Verse"
        .Columns("Content").HeaderText = "Content"
        .Columns("Translation").HeaderText = "Translation"
        .Columns("Remarks").HeaderText = "Remarks"
        .Columns("Category").HeaderText = "Category"
    End With
End Sub

Private Sub addButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
addButton.Click
    Try
        If Not IsValidInput() Then Return
        Dim row As DataRow = store.NewRow()
        row.Item("VerseID") = nextId
        row.Item("Book") = txtBook.Text.Trim()
        row.Item("Chapter") = IntValue(txtChapter.Text)
        row.Item("Verse") = IntValue(txtVerse.Text)
        row.Item("Content") = txtContent.Text.Trim()
        row.Item("Translation") = txtTranslation.Text.Trim()
        row.Item("Remarks") = txtRemarks.Text.Trim()
        row.Item("Category") = txtCategory.Text.Trim()
        store.Rows.Add(row)
        nextId += 1
        SaveStore()
        MessageBox.Show("The record was saved." & vbCrLf & vbCrLf & "New ID: " & (nextId -
1).ToString(), _
            "Record Added", MessageBoxButtons.OK, MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Add failed: " & ex.Message)
        MessageBox.Show("The record could not be saved." & vbCrLf & vbCrLf & ex.Message, _
            "Save Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub updateButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
updateButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If Not IsValidInput() Then Return
        Dim found As DataRow = Nothing
        For Each row As DataRow In store.Rows
            If Convert.ToInt32(row.Item("VerseID")) = selectedId Then
                found = row
                Exit For
            End If
        Next
        If found Is Nothing Then
            MessageBox.Show("That record no longer exists.", "Record Not Found", _
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            RefreshGrid()
            Return
        End If
        found.Item("Book") = txtBook.Text.Trim()
        found.Item("Chapter") = IntValue(txtChapter.Text)
        found.Item("Verse") = IntValue(txtVerse.Text)
        found.Item("Content") = txtContent.Text.Trim()
        found.Item("Translation") = txtTranslation.Text.Trim()
        found.Item("Remarks") = txtRemarks.Text.Trim()
        found.Item("Category") = txtCategory.Text.Trim()
```

```
        SaveStore()
        MessageBox.Show("The record was updated.", "Record Updated", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Update failed: " & ex.Message)
        MessageBox.Show("The record could not be updated." & vbCrLf & vbCrLf & ex.Message, _
            "Update Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub deleteButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
deleteButton.Click
    Try
        If selectedId = 0 Then
            MessageBox.Show("Select a record in the table first.", "No Record Selected", _
                MessageBoxButtons.OK, MessageBoxIcon.Information)
            Return
        End If
        If MessageBox.Show("Delete this record?" & vbCrLf & vbCrLf & "ID: " & selectedId.ToString(), _
                "Confirm Delete", MessageBoxButtons.YesNo, MessageBoxIcon.Question) <> DialogResult.Yes
Then
            Return
        End If
        For i As Integer = store.Rows.Count - 1 To 0 Step -1
            If Convert.ToInt32(store.Rows(i).Item("VerseID")) = selectedId Then
                store.Rows.RemoveAt(i)
            End If
        Next
        SaveStore()
        MessageBox.Show("The record was deleted.", "Record Deleted", MessageBoxButtons.OK,
MessageBoxIcon.Information)
        ClearFields()
        RefreshGrid()
    Catch ex As Exception
        LogStep("Delete failed: " & ex.Message)
        MessageBox.Show("The record could not be deleted." & vbCrLf & vbCrLf & ex.Message, _
            "Delete Error", MessageBoxButtons.OK, MessageBoxIcon.Warning)
    End Try
End Sub

Private Sub clearButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
clearButton.Click
    ClearFields()
End Sub

Private Sub exitButton_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
exitButton.Click
    Me.Close()
End Sub

Private Sub dataGridView1_SelectionChanged(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles dataGridView1.SelectionChanged
    If isLoading Then Return
    If dataGridView1.CurrentRow Is Nothing Then Return
    FillFromRow()
End Sub

Private Sub FillFromRow()
    Dim row As DataGridViewRow = dataGridView1.CurrentRow
    If row Is Nothing Then Return
    If row.Cells("VerseID").Value Is Nothing Then Return
    If row.Cells("VerseID").Value Is DBNull.Value Then Return
    isLoading = True
    Try
        selectedId = Convert.ToInt32(row.Cells("VerseID").Value)
        txtVerseID.Text = selectedId.ToString()
        txtBook.Text = Convert.ToString(row.Cells("Book").Value)
        txtChapter.Text = Convert.ToString(row.Cells("Chapter").Value)
        txtVerse.Text = Convert.ToString(row.Cells("Verse").Value)
        txtContent.Text = Convert.ToString(row.Cells("Content").Value)
        txtTranslation.Text = Convert.ToString(row.Cells("Translation").Value)
        txtRemarks.Text = Convert.ToString(row.Cells("Remarks").Value)
        txtCategory.Text = Convert.ToString(row.Cells("Category").Value)
    Catch ex As Exception
        LogStep("FillFromRow failed: " & ex.Message)
    Finally
        isLoading = False
    End Try
End Sub

Private Sub ClearFields()
    isLoading = True
    Try
        dataGridView1.CurrentCell = Nothing
    Catch
```

```
        End Try
        selectedId = 0
        txtVerseID.Clear()
        txtBook.Clear()
        txtChapter.Clear()
        txtVerse.Clear()
        txtContent.Clear()
        txtTranslation.Clear()
        txtRemarks.Clear()
        txtCategory.Clear()
        txtBook.Focus()
        isLoading = False
    End Sub

    Private Function IsValidInput() As Boolean
        If txtBook.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Book.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtBook.Focus()
            Return False
        End If
        If txtChapter.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Chapter.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtChapter.Focus()
            Return False
        End If
        Dim ChapterNum As Integer = 0
        If txtChapter.Text.Trim() <> "" AndAlso Not Integer.TryParse(txtChapter.Text.Trim(), ChapterNum)
    Then
            MessageBox.Show("The Chapter must be a whole number.", "Invalid Number", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtChapter.Focus()
            Return False
        End If
        If txtVerse.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Verse.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtVerse.Focus()
            Return False
        End If
        Dim VerseNum As Integer = 0
        If txtVerse.Text.Trim() <> "" AndAlso Not Integer.TryParse(txtVerse.Text.Trim(), VerseNum) Then
            MessageBox.Show("The Verse must be a whole number.", "Invalid Number", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtVerse.Focus()
            Return False
        End If
        If txtContent.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Content.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtContent.Focus()
            Return False
        End If
        If txtTranslation.Text.Trim() = "" Then
            MessageBox.Show("Please enter the Translation.", "Missing Information", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation)
            txtTranslation.Focus()
            Return False
        End If
        Return True
    End Function


    Private Function IntValue(ByVal text As String) As Integer
        Dim value As Integer = 0
        If Integer.TryParse(text.Trim(), value) Then Return value
        Return 0
    End Function

    Private Function DateValue(ByVal text As String) As Date
        Dim parts As String() = text.Split("-"c)
        If parts.Length = 3 Then
            Dim y As Integer = 0
            Dim m As Integer = 0
            Dim d As Integer = 0
            If Integer.TryParse(parts(0), y) AndAlso Integer.TryParse(parts(1), m) AndAlso
    Integer.TryParse(parts(2), d) Then
                If y >= 1 AndAlso m >= 1 AndAlso m <= 12 AndAlso d >= 1 AndAlso d <= 31 Then
                    Return New Date(y, m, d)
                End If
            End If
        End If
        Return Date.Today
    End Function

End Class
```

frmAbout.vb

```
 Public Class frmAbout

     Private Sub LogStep(ByVal text As String)
         Try
             IO.File.AppendAllText(IO.Path.Combine(Application.StartupPath, "forms_log.txt"), _
                 Date.Now.ToString("HH:mm:ss") & "  [frmAbout] " & text & vbCrLf)
         Catch
         End Try
     End Sub

     Private Const PhotoData As String = _
         "/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUDBAQEAwUEBAQFBQUGBwwIBwcHBw8LCwkMEQ8SEhEPERETFhwXExQaFRERGCEYGh0
     dHx8fExciJCI ...
 ' [very long data line omitted in this document]

     Private Sub frmAbout_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
     MyBase.Load
         LogStep("Load start")
         Try
             Dim bytes As Byte() = Convert.FromBase64String(PhotoData)
             Using stream As New IO.MemoryStream(bytes)
                 Using photo As Image = Image.FromStream(stream)
                     PictureBox1.Image = New Bitmap(photo)
                 End Using
             End Using
         Catch ex As Exception
         End Try
     LogStep("Load end")
     End Sub

 End Class
```

Notes

The long Base64 picture data inside frmAbout.vb is left out of this document so that the source listing stays readable. The complete file is inside the IntegProgProject2026 folder of the compilation.

Appendix A - Project file list

Every folder below is inside the COMPILATION folder of the submitted archive. The bin, obj and .suo files of the original projects are not included.

Act1_Activity202606241600

```
Act1_Activity202606241600/Activity202606241600.sln
Act1_Activity202606241600/Activity202606241600/Activity202606241600.vbproj
Act1_Activity202606241600/Activity202606241600/Module1.vb
```

Act2 Celcius-Fahrenheit Converter

```
Act2 Celcius-Fahrenheit Converter/Act2.sln
Act2 Celcius-Fahrenheit Converter/Act2/Act2.vbproj
Act2 Celcius-Fahrenheit Converter/Act2/Module1.vb
```

Activity3_20260629

```
Activity3_20260629/Activity3_20260629.sln
Activity3_20260629/Activity3_20260629/Activity3_20260629.vbproj
Activity3_20260629/Activity3_20260629/Module1.vb
```

Activity4_20260703

```
Activity4_20260703/Activity4_20260703.sln
Activity4_20260703/Activity4_20260703/Activity4_20260703.vbproj
Activity4_20260703/Activity4_20260703/Form1.Designer.vb
Activity4_20260703/Activity4_20260703/Form1.vb
```

Act5 - Ternary2

```
Act5/Ternary2.sln
Act5/Ternary2/Form1.Designer.vb
Act5/Ternary2/Form1.vb
Act5/Ternary2/Ternary2.vbproj
```

Act6_IntegProg1_20260715

```
Act6_IntegProg1_20260715/IntegrativeProg1_20260715.sln
Act6_IntegProg1_20260715/IntegrativeProg1_20260715/Form1.Designer.vb
Act6_IntegProg1_20260715/IntegrativeProg1_20260715/Form1.vb
Act6_IntegProg1_20260715/IntegrativeProg1_20260715/IntegrativeProg1_20260715.vbproj
```

Act7_IntegProg1_SW20260720

```
Act7_IntegProg1_SW20260720/IntegProg1_SW20260720.sln
Act7_IntegProg1_SW20260720/IntegProg1_SW20260720/Form1.Designer.vb
Act7_IntegProg1_SW20260720/IntegProg1_SW20260720/Form1.vb
Act7_IntegProg1_SW20260720/IntegProg1_SW20260720/IntegProg1_SW20260720.vbproj
```

Act8_SW20260805

```
Act8_SW20260805/TheMessage.sln
Act8_SW20260805/TheMessage/Form1.Designer.vb
Act8_SW20260805/TheMessage/Form1.vb
Act8_SW20260805/TheMessage/TheMessage.vbproj
```

SW202609301600

SW202609301600/ListViewSample.Designer.vb

SW202609301600/ListViewSample.vb

SW202609301600/SW202609301600.sln

SW202609301600/SW202609301600.vbproj

SW20261002 - PanelSplitPanelDemo

SW20261002/PanelSplitPanelDemo.sln

SW20261002/PanelSplitPanelDemo/Form1.Designer.vb

SW20261002/PanelSplitPanelDemo/Form1.vb

SW20261002/PanelSplitPanelDemo/Form2.Designer.vb

SW20261002/PanelSplitPanelDemo/Form2.vb

SW20261002/PanelSplitPanelDemo/PanelSplitPanelDemo.vbproj

SW202610021600

SW202610021600/Application.vb

SW202610021600/CombinedSample.Designer.vb

SW202610021600/CombinedSample.vb

SW202610021600/ListViewSample.Designer.vb

SW202610021600/ListViewSample.vb

SW202610021600/MainForm.vb

SW202610021600/MenuForm.vb

SW202610021600/OpenFileDialogSample.Designer.vb

SW202610021600/OpenFileDialogSample.vb

SW202610021600/SW202610021600.sln

SW202610021600/SW202610021600.vbproj

SW202610021600/TreeViewSample.Designer.vb

SW202610021600/TreeViewSample.vb

FileExplorerClone - PanelSplitDemo

FileExplorerClone/PanelSplitDemo.sln

FileExplorerClone/PanelSplitDemo/Form1.Designer.vb

FileExplorerClone/PanelSplitDemo/Form1.vb

FileExplorerClone/PanelSplitDemo/PanelSplitDemo.vbproj

IntegProgProject2026

IntegProgProject2026/IntegProgProject2026.sln

IntegProgProject2026/IntegProgProject2026/IntegProgProject2026.vbproj

IntegProgProject2026/IntegProgProject2026/Forms/frmAbout.Designer.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmAbout.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmBibleVerses.Designer.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmBibleVerses.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmMain.Designer.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmMain.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmQuotes.Designer.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmQuotes.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmSchedules.Designer.vb

IntegProgProject2026/IntegProgProject2026/Forms/frmSchedules.vb

Appendix B - What changed in this compilation

Author and course - The name Alvarez, Rex was replaced by Mangalili, Fhetz and the course line now reads BSIT 3B in every project, every form title, the About form, the application configuration file and the console programs.

Activity 4 - New two panel design with a dark banner, a shared ReadInputs() function, four decimal result formatting and a division by zero guard.

Activity 5 - New side rail layout with three result cards, the Enter key runs the evaluation and the first card changes colour with the result.

Activity 6 - New left card and right result layout with a large average badge, one ReadGrades() validation function and a 0 to 100 range check.

Activity 8 - The four overlapping GroupBoxes were replaced by a compose card and three option cards; the long If chain was replaced by helper functions.

Activity 10 - Both forms were rebuilt: Panel Playground with an event log and an API inspector, and Split Container Lab with a category list, a record list, a details pane and a reset layout feature.

Activity 12 - The explorer was rewritten as Mangalili Explorer with history stacks, an address bar, a live filter, numeric column sorting, safe image preview and layout buttons for the nested splitters.

Prepared by : Mangalili, Fhetz Course : BSIT 3B Date : October 26, 2026